



**POLITECNICO**  
**MILANO 1863**

SCUOLA DI INGEGNERIA INDUSTRIALE  
E DELL'INFORMAZIONE

# Numerical Study of Turbulent Flows: Diffuser and NACA4415 Airfoil

059697 COMPUTATIONAL FLUID DYNAMICS - FUNDAMENTALS

Group 21:

Luca Massari 10840428 (315135)

Fabio Marchi 10813048 (274367)

Mattia Bragonzi 10830867 (315278)

Anno Accademico: 2025-26

## Abstract

This report presents a computational fluid dynamics (CFD) analysis of two engineering flow problems: turbulent flow through a diffuser and external flow around an airfoil (NACA4415). The simulations were performed using OpenFOAM, investigating the effects of mesh refinement, turbulence modelling, and near wall treatment on the numerical results.

For the diffuser case, a grid independence study was conducted using three mesh resolutions. The results showed good agreement between the medium and fine meshes and correctly captured the expected flow physics, including velocity reduction, pressure recovery, and total pressure losses. Additional simulations performed without wall functions confirmed the accuracy and robustness of the numerical solution. In particular, a velocity reduction coefficient of approximately 0.84 was obtained, confirming the presence of irreversible energy losses within the diffuser.

For the airfoil case, the aerodynamic behaviour was analyzed at different angles of attack using both the Spalart–Allmaras and  $k-\varepsilon$  turbulence models. The comparison of lift and drag coefficients, pressure distributions, wall shear stress, and wake profiles demonstrated grid-independent and physically consistent results. A good agreement was observed between the different turbulence models and wall treatments, providing confidence in the reliability of the simulations.

# Contents

|                        |  |
|------------------------|--|
| Introduction . . . . . |  |
|------------------------|--|

## Contents

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Simulation of the 2D incompressible flow within a divergent channel</b>  | <b>1</b>  |
| 1.1      | Introduction . . . . .                                                      | 1         |
| 1.2      | Geometrical Domain . . . . .                                                | 2         |
| 1.3      | Simulation with wall functions . . . . .                                    | 2         |
| 1.3.1    | Mesh Generation and Grid Independence Study . . . . .                       | 2         |
| 1.3.2    | $k-\omega$ SST model . . . . .                                              | 4         |
| 1.3.3    | Results . . . . .                                                           | 9         |
| 1.4      | Simulation resolving the flow up to the wall . . . . .                      | 13        |
| 1.4.1    | Mesh Generation . . . . .                                                   | 13        |
| 1.4.2    | $k-\omega$ SST model . . . . .                                              | 14        |
| 1.4.3    | SA model . . . . .                                                          | 16        |
| 1.4.4    | Results . . . . .                                                           | 18        |
| 1.5      | Conclusions . . . . .                                                       | 20        |
| <b>2</b> | <b>Simulation of the 2D incompressible flow around the NACA4415 airfoil</b> | <b>21</b> |
| 2.1      | Introduction . . . . .                                                      | 21        |
| 2.2      | Geometrical Domain . . . . .                                                | 21        |
| 2.3      | Simulation with wall functions . . . . .                                    | 22        |
| 2.3.1    | Mesh Generation and Grid Independence Study . . . . .                       | 22        |
| 2.3.2    | SA model . . . . .                                                          | 25        |
| 2.3.3    | $k-\varepsilon$ model . . . . .                                             | 28        |
| 2.3.4    | Results . . . . .                                                           | 30        |
| 2.4      | Simulation resolving the flow up to the wall . . . . .                      | 35        |
| 2.4.1    | Mesh Generation . . . . .                                                   | 35        |
| 2.4.2    | SA model with AOA=5° . . . . .                                              | 36        |
| 2.4.3    | SA model with AOA=0° . . . . .                                              | 37        |
| 2.4.4    | Results . . . . .                                                           | 38        |
| 2.5      | Conclusions . . . . .                                                       | 40        |

# 1 | Simulation of the 2D incompressible flow within a divergent channel

## 1.1. Introduction

The present work concerns the numerical simulation of the two-dimensional incompressible flow within a divergent channel, carried out using the open-source CFD solver OpenFOAM. Two turbulence models employing wall functions are considered, and the analysis includes a grid independence study based on three progressively refined meshes. An additional set of simulations is performed using wall-resolving turbulence models, without the use of wall functions, in order to assess the impact of near-wall modeling on the solution.

Since the flow is incompressible and fluid properties are assumed constant, the velocity and pressure fields are governed only by the continuity and momentum equations, and the energy equation is not required to close the system. The governing equations therefore reduce to the incompressible Navier-Stokes equations.

The simulation is two-dimensional: no variation of flow quantities is assumed along the  $z$  direction, and a single cell layer is used in that direction, as required by OpenFOAM.

The key performance indicators (KPIs) evaluated in both cases are:

1. flow rate and velocity reduction coefficient at the channel exit, defined as  $\varphi = V_{\text{OUT}}/V_{\text{OUT,IS}}$ ;
2. longitudinal distribution of cross-sectional-averaged total and static pressure along the duct;
3. distribution of pressure and wall shear stress along the channel solid boundary.

## 1.2. Geometrical Domain

The channel geometry consists of three sections in series: a straight inlet channel, a diverging section, and a straight outlet channel. The fluid is air at ambient conditions, with kinematic viscosity  $\nu = 1.5 \times 10^{-5} \text{ m}^2/\text{s}$  and density  $\rho = 1.2 \text{ kg/m}^3$ .

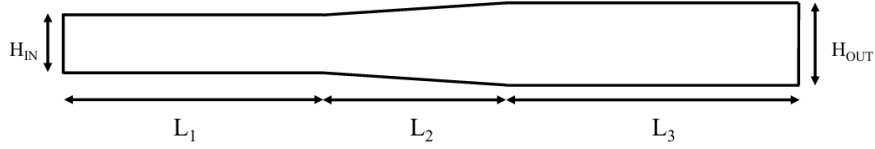


Figure 1.1: Divergent channel geometry.

The main geometric parameters are summarised in Table 1.1.

| $L_1$ | $L_2$ | $L_3$ | $H_{\text{in}}$ | $H_{\text{out}}$ |
|-------|-------|-------|-----------------|------------------|
| 0.7 m | 1.0 m | 1.0 m | 0.125 m         | 0.25 m           |

Table 1.1: Divergent channel dimensions.

The boundary conditions are: uniform inlet velocity with turbulence intensity  $I_{\text{in}} = 5\%$ , turbulence length scale  $l_{\text{in}} = 0.025 \text{ m}$  ( $1/10 H_{\text{in}}$ ), and a  $\Delta P = 129.34 \text{ Pa}$  which is equal to the difference between the total pressure at the inlet and the static pressure at the outlet.

## 1.3. Simulation with wall functions

### 1.3.1. Mesh Generation and Grid Independence Study

To generate the mesh, a sequence of OpenFOAM utilities was employed.

First, the *blockMesh* tool was used to create a structured three-dimensional rectangular mesh matching the overall dimensions of the divergent channel. Subsequently, *snappyHexMesh* was used to generate the divergent section of the channel and to add the boundary-layer refinement.

Once the three-dimensional mesh was completed, it was converted into a two-dimensional mesh using the *extrudeMesh* tool. For the simulation, the front and back boundaries must be defined as *empty* patches. However, to ensure the correct execution of *snappyHexMesh*, these boundaries initially need to be specified as *wall* patches. Therefore, the final step consisted of converting the front and back boundaries from *wall* to *empty* using the *createPatch* tool.

The first requirement to be addressed is the grid independence study, performed using three progressively refined meshes. The refinement was achieved by increasing the number of cells in the *blockMesh* tool and, when necessary, adjusting the boundary-layer settings in *snappyHexMesh*.

It is important to note that the mesh refinement was performed with the specific objective of maintaining a  $y^+$  value  $\geq 30$ , which is the valid range for simulations utilizing wall functions.

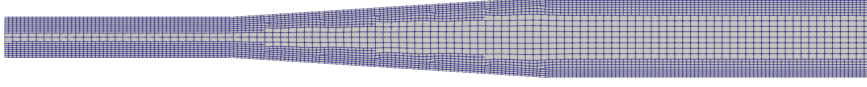


Figure 1.2: Mesh 1: low refinement

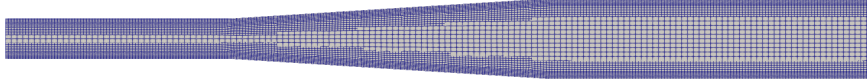


Figure 1.3: Mesh 2: mid refinement

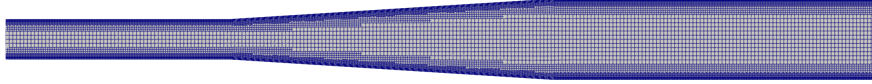


Figure 1.4: Mesh 3: high refinement

After the mesh generation process, a quality check was performed on each mesh using the *checkMesh* utility of OpenFOAM. The assessment considered the number of cells, the maximum aspect ratio, the maximum skewness, the average non-orthogonality and the value of the non-dimensional wall distance ( $y^+$ ).

The mesh-quality metrics for the three grids are summarized in Table 1.2 and Table 1.3.

|             | N. of cells | Max aspect ratio | Max skewness | Av. non-orthogonality |
|-------------|-------------|------------------|--------------|-----------------------|
| <b>low</b>  | 5828        | 3.809            | 0.286        | 1.866                 |
| <b>mid</b>  | 10530       | 4.053            | 0.285        | 1.665                 |
| <b>high</b> | 20250       | 2.983            | 0.292        | 1.322                 |

Table 1.2: Mesh quality metrics

|             | Min $y^+$ | Average $y^+$ | Max $y^+$ |
|-------------|-----------|---------------|-----------|
| <b>low</b>  | 102.918   | 146.442       | 229.781   |
| <b>mid</b>  | 53.613    | 88.151        | 153.673   |
| <b>high</b> | 27.566    | 40.340        | 94.721    |

Table 1.3: Values of  $y^+$

The minimum value of  $y^+$  for the high refinement mesh is slightly below 30. However, this is acceptable because it occurs only in a small isolated region and the Average value remains in the recommended range.

### 1.3.2. $k$ - $\omega$ SST model

#### Boundary Conditions

The velocity and pressure boundary conditions are shown in Fig. 1.5 and are kept constant among the three simulations for the grid independence study.

```
FoamFile
{
    format      ascii;
    class       volVectorField;
    object       U;
}
// .....

dimensions      [0 1 -1 0 0 0];

internalField    uniform (14.7 0 0);

boundaryField
{
    inlet
    {
        type      pressureDirectedInletVelocity;
        inletDirection    uniform (1 0 0);
        value       uniform (14.7 0 0);
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      noSlip;
    }

    divergent_bottomWall
    {
        type      noSlip;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}

FoamFile
{
    format      ascii;
    class       volScalarField;
    object       p;
}
// .....

dimensions      [0 2 -2 0 0 0];

internalField    uniform 107.7833;

boundaryField
{
    inlet
    {
        type      totalPressure;
        p0         uniform 107.7833;
    }

    outlet
    {
        type      fixedValue;
        value       uniform 0;
    }

    divergent_topWall
    {
        type      zeroGradient;
    }

    divergent_bottomWall
    {
        type      zeroGradient;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

Figure 1.5: Boundary conditions: velocity (left) and pressure (right).

The boundary conditions are defined to ensure a consistent and physically accurate RANS approach for turbulent internal flow.

To perform the simulation the  $\Delta P$  was given and was equal to the difference of total pressure at the inlet and static pressure at the outlet and it was  $\Delta P=129.34$  Pa. According to OpenFoam, incompressible solvers work with kinematic pressure, so pressure was divided by the density.

The choice made was to consider  $P_{outlet}$  as 0, imposing a *fixedValue* condition, in order to consider the  $\Delta P = P_{inlet}$ , and so impose at the inlet a *totalPressure* condition. This combination establishes a standard, well-posed boundary configuration for internal fluid flows. Along the physical walls, a *zeroGradient* condition is utilized, allowing the pressure field to adjust naturally without introducing unphysical kinematic constraints.

Velocity was initially estimated to provide a starting value for the computations, according to the formula:

$$U_{inlet} = \sqrt{\frac{2 \Delta P}{\rho}} \quad (1.1)$$

At the inlet patch is applied a *pressureDirectedInletVelocity* boundary condition, while at the outlet, a *zeroGradient* condition allows the velocity profile to adjust freely according to the internal solution. A *noSlip* condition is enforced on the top and bottom walls to account for viscous wall friction, whereas the front and back patches are assigned as *empty*, effectively modeling a 2D computational domain.

The  $k$ ,  $w$  and  $\nu_T$  boundary conditions are shown in Fig. 1.6 and kept constant among the three simulations for the grid independence study.

For the turbulent kinetic energy  $k$ , the domain is uniformly initialized with  $k = 0.81$ , initially estimated via the analytical relation:

$$k = \frac{3}{2}(UI)^2$$

Where  $I$  represents the turbulence intensity (provided by the problem specifications). At the inlet patch, a *fixedValue* boundary condition is applied, while the outlet employs a *zeroGradient* formulation. The wall boundaries are governed by the *kqRWallFunction*, consistent with high-Reynolds near-wall turbulence modeling. The specific dissipation rate  $\omega$  is initialized uniformly across the computational domain with a value of  $\omega = 65.7267 \text{ s}^{-1}$ , calculated via the analytical empirical formula:

$$\omega = C_\mu^{-\frac{1}{4}} \frac{\sqrt{k}}{l_T}$$

Where  $C_\mu = 0.09$  is a standard empirical constant of the  $k - \omega$  model, and  $l_T$  represents the turbulent length scale. Regarding the boundary conditions, a *fixedValue* condition is applied at the inlet patch, whereas the outlet employs a *zeroGradient* formulation. Along the solid boundaries, the *omegaWallFunction* is implemented. In accordance with the 2D domain assumption, the front and back patches are configured as *empty*. Finally, the kinematic turbulent viscosity  $\nu_T$  is initialized uniformly with a value of  $\nu_T = 0.012323 \text{ m}^2/\text{s}$ , derived directly from the constitutive relation:

$$\nu_T = \frac{k}{\omega}$$

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    location    "0";
    object      k;
}
// *****

dimensions      [0 2 -2 0 0 0 0];

inletk          0.81;

internalField   uniform $inletk;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform $inletk;
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }

    divergent_bottomWall
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}

```

(a) Turbulent kinetic energy boundary condition

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    location    "0";
    object      omega;
}
// *****

dimensions      [0 0 -1 0 0 0 0];

inletOmega      65.7267;

internalField   uniform $inletOmega;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform $inletOmega;
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      omegaWallFunction;
        value      uniform $inletOmega;
    }

    divergent_bottomWall
    {
        type      omegaWallFunction;
        value      uniform $inletOmega;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}

```

(b) Omega boundary condition

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    location    "0";
    object      nut;
}
// *****

dimensions      [0 2 -1 0 0 0 0];

internalField   uniform 0.012323;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.012323;
    }

    outlet
    {
        type      zeroGradient;
        value      uniform 0.012323;
    }

    divergent_topWall
    {
        type      nutkWallFunction;
        value      uniform 0.012323;
    }

    divergent_bottomWall
    {
        type      nutkWallFunction;
        value      uniform 0.012323;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}

```

(c) Turbulent Kinematic Viscosity boundary condition

Figure 1.6: Boundary condition for  $k$ ,  $w$  and  $nu_T$

## Discretization Scheme and Solver

Regarding the numerical discretization schemes, Fig. 1.7a illustrates the specific methods adopted to approximate the spatial and temporal operators within the governing equations:



Figure 1.7: Discretization schemes and solver

In turbulent flow simulations, the selection of appropriate numerical schemes is critical to ensuring both solution stability and robust convergence. Turbulence introduces steep spatial gradients and complex non-linear interactions, which demand careful treatment during the discretization process.

To achieve second-order accuracy while preserving numerical stability, the convective term for velocity ( $\text{div}(\phi, \mathbf{U})$ ) is discretized using a *bounded Gauss linearUpwind grad(U)* scheme. This approach introduces a controlled amount of numerical diffusion strictly in regions of high gradients without significantly compromising the overall spatial accuracy of the solution.

Instead, for the transport equations of the turbulent kinetic energy ( $k$ ) and the specific dissipation

ation rate ( $\omega$ ), a first-order upwind scheme (*bounded Gauss upwind*) is employed. This choice represents a robust and standard practice in *Reynolds-Averaged Navier-Stokes* (RANS) simulations; the inherent numerical boundedness of the upwind scheme prevents unphysical, negative values of  $k$  and  $\omega$ , thereby stabilizing the iterative solver in the presence of severe near-wall gradients.

The linear solvers, tolerance criteria, and algorithms selected for the pressure and velocity fields, as well as the turbulence quantities, are detailed in Fig. 1.7b.

For both the pressure field ( $p$ ) and the turbulent quantities (" $k|\omega$ "), a *Geometric-Algebraic Multigrid* (GAMG) solver is employed. This choice is dictated by its superior robustness and computational efficiency, making it highly effective at accelerating convergence when solving large, coupled systems of equations on unstructured meshes. Conversely, the velocity field ( $\mathbf{U}$ ) is resolved utilizing a *smoothSolver* coupled with *GaussSeidel* smoothing.

To guarantee numerical stability during the iterative execution of the steady-state SIMPLE algorithm, appropriate under-relaxation factors are enforced. Specifically, the relaxation factor is set to 0.3 for pressure, 0.7 for velocity, and 0.6 for the turbulence variables  $k$  and  $\omega$ . These specific coefficients establish an optimal compromise between iterative stability and convergence rate, effectively dampening numerical oscillations within the coupled system without introducing excessive under-relaxation.

### 1.3.3. Results

The first aspect to be verified is the correct convergence of the residuals. As shown in Fig. 1.8, all simulations reached the target residual value of  $1e-5$ .

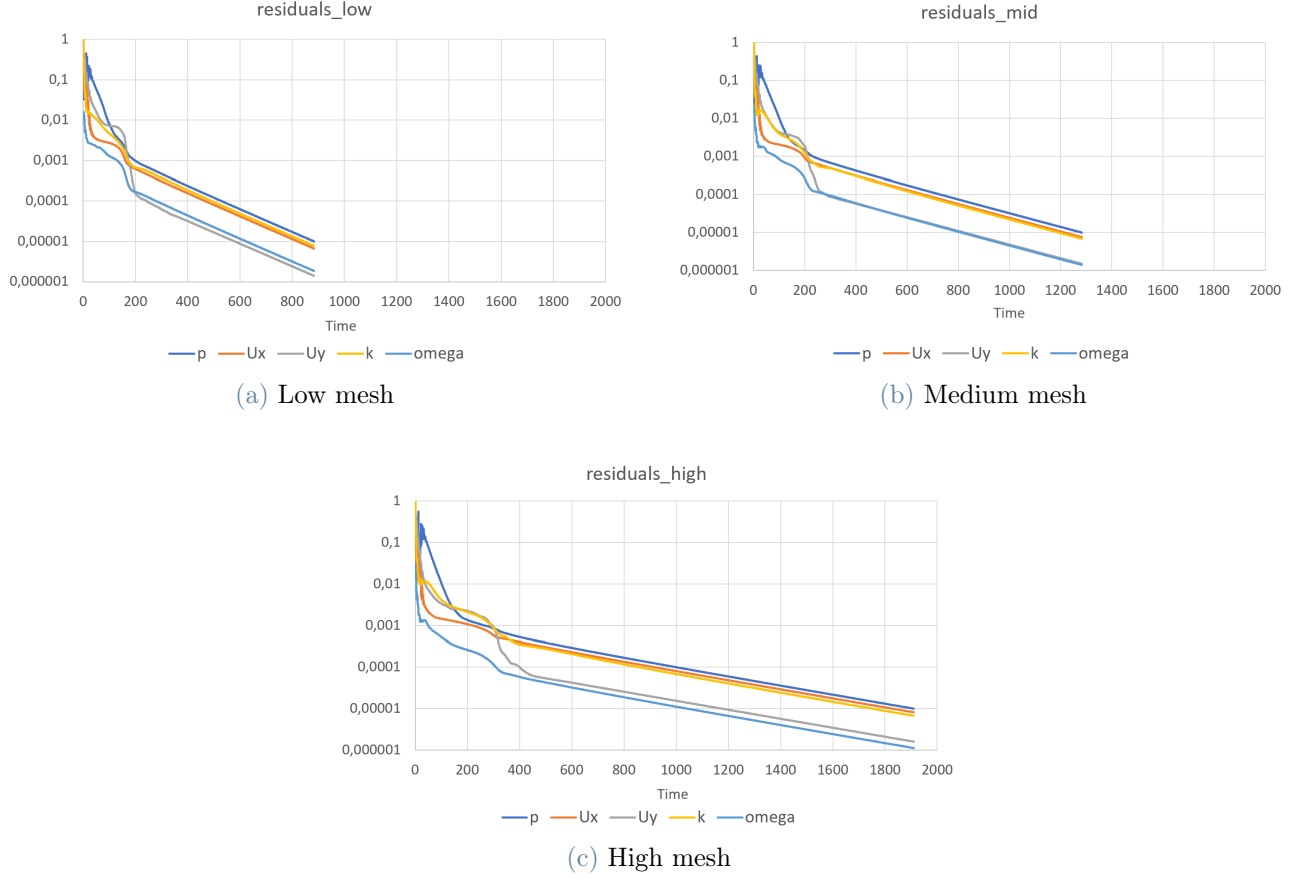


Figure 1.8: Residual for the three mesh resolutions.

As the mesh refinement increases, a larger number of iterations is required to achieve convergence. This behaviour is expected, since mesh refinement generally leads to an increase in the condition number. However, the residual curves remain stable in all the simulations, indicating that the solver settings are appropriate and that all cases have a good convergence.

The following pages present the values and plots of the required KPIs. For each KPI, a comparison among the three mesh resolutions is provided in order to assess the effect of mesh refinement on the results and to evaluate the grid independence of the solution.

In the following table, the values of the flow rate and the velocity reduction coefficient at the channel exit are reported.

|             | flow rate, $[m^3/s]$ | $\phi$ |
|-------------|----------------------|--------|
| <b>low</b>  | 0.240                | 0.817  |
| <b>mid</b>  | 0.247                | 0.842  |
| <b>high</b> | 0.249                | 0.847  |

Table 1.4: flow rate and velocity reduction coefficients

The small difference between the results obtained with the medium and the high refined meshes confirms that the grid independence has been reached.

The velocity reduction coefficient remains below unity, so the diffuser does not achieve ideal isentropic pressure recovery. This is due to the viscous dissipation.

Figures 1.9, 1.10, and 1.11 show, respectively, the cross-sectional averages of velocity, static pressure, and total pressure. For each quantity, the average value was computed over a series of cross-sections along the channel. A total of 30 cross-sections were considered to obtain a clear and representative description of the variation of these quantities throughout the channel.

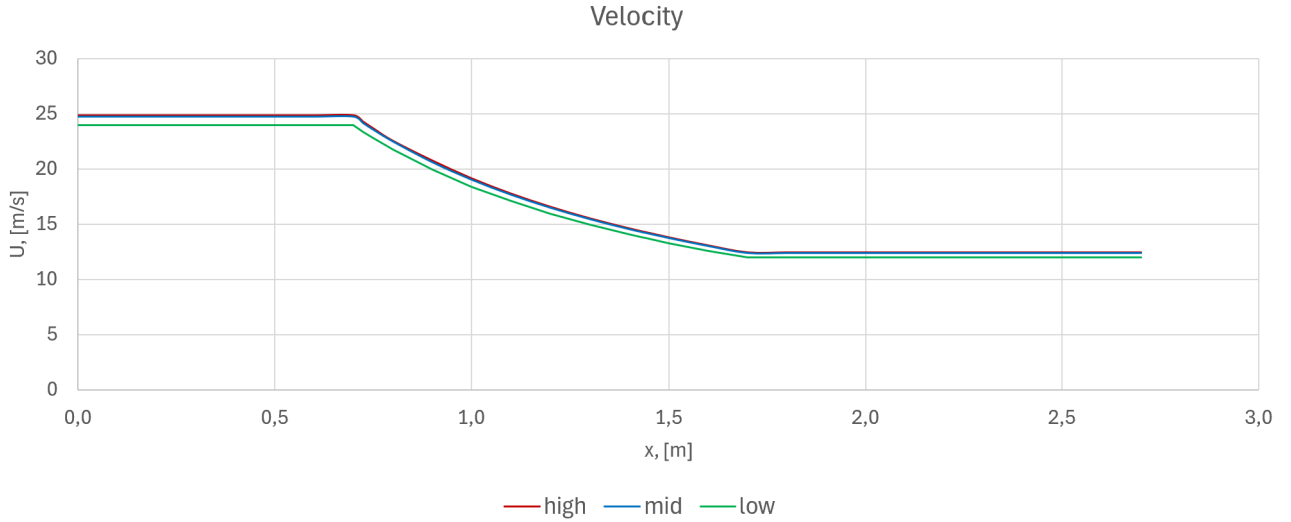


Figure 1.9: Cross-sectional averaged velocity profile

The outlet velocity is approximately half of the inlet velocity. This result is consistent with the conservation of mass, since the diffuser outlet height is twice the inlet height.

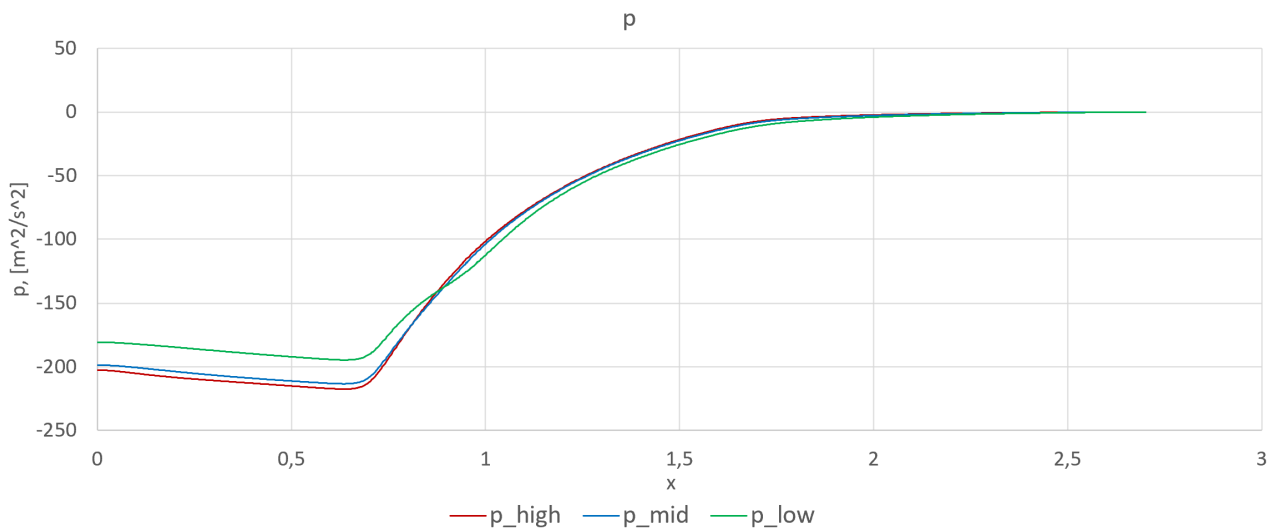


Figure 1.10: Static pressure distribution along the channel

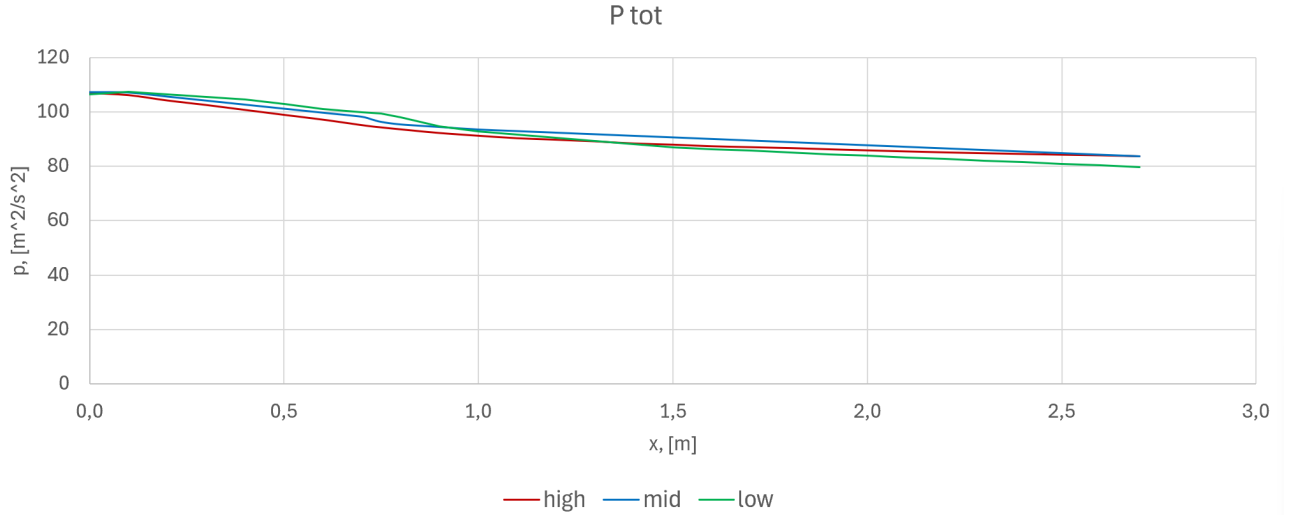


Figure 1.11: Total pressure distribution along the channel

For the static pressure, a small decrease is observed in the first section of the channel. This reduction is caused by viscous friction at the walls, which leads to pressure losses. This behaviour becomes less evident in the last section because the velocity decreases while the channel height increases, so there is a reduction of the wall shear stress and consequently of the friction losses.

In the divergent section, a significant increase in static pressure is observed. This is due to the reduction in flow velocity, which results in a conversion of kinetic energy into static pressure.

This pressure profile reflects the expected physics of a divergent channel. The total pressure also reflects the expected physics. There is a higher slope at the inlet where the higher velocity leads to greater viscous dissipation and then with the reduction of velocity the slope decreases. The velocity and static pressure profiles obtained with the medium and high refined meshes show an excellent agreement, confirming that grid independence has been reached for these quantities. On the other hand, a larger discrepancy is observed in the total pressure distribution. This can be attributed to the fact that total pressure depends on both static pressure and velocity, making it more sensitive to small numerical differences between the meshes. In addition, the slight differences in the predicted flow rate for the low and medium meshes may contribute to the observed deviations in total pressure.

For the analysis of pressure and wall shear stress distributions along the channel boundary, the top wall was selected as the reference surface.

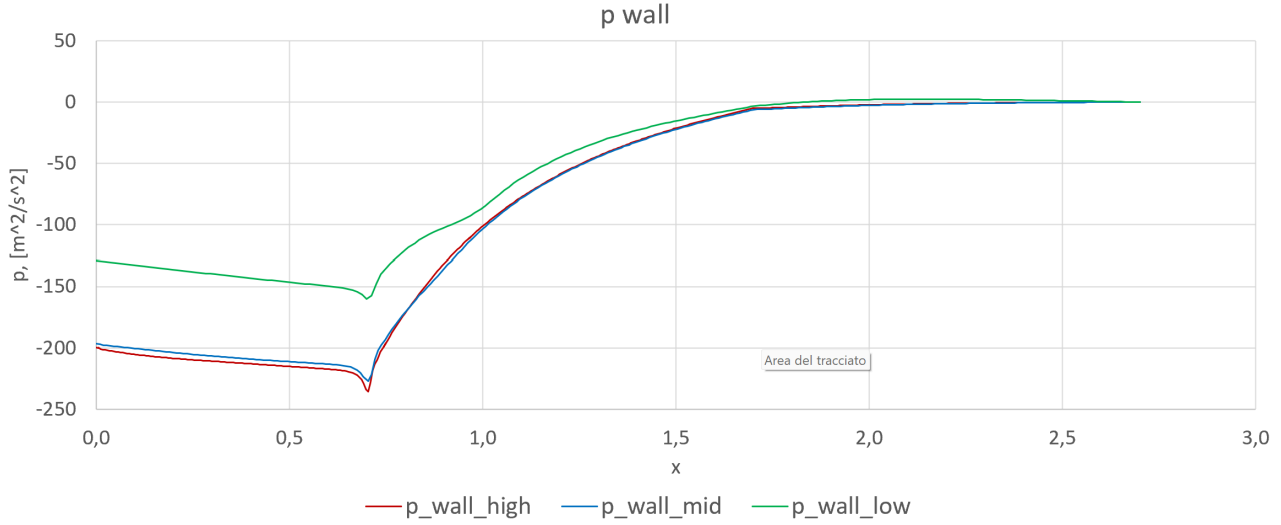


Figure 1.12: Pressure distribution along the wall channel

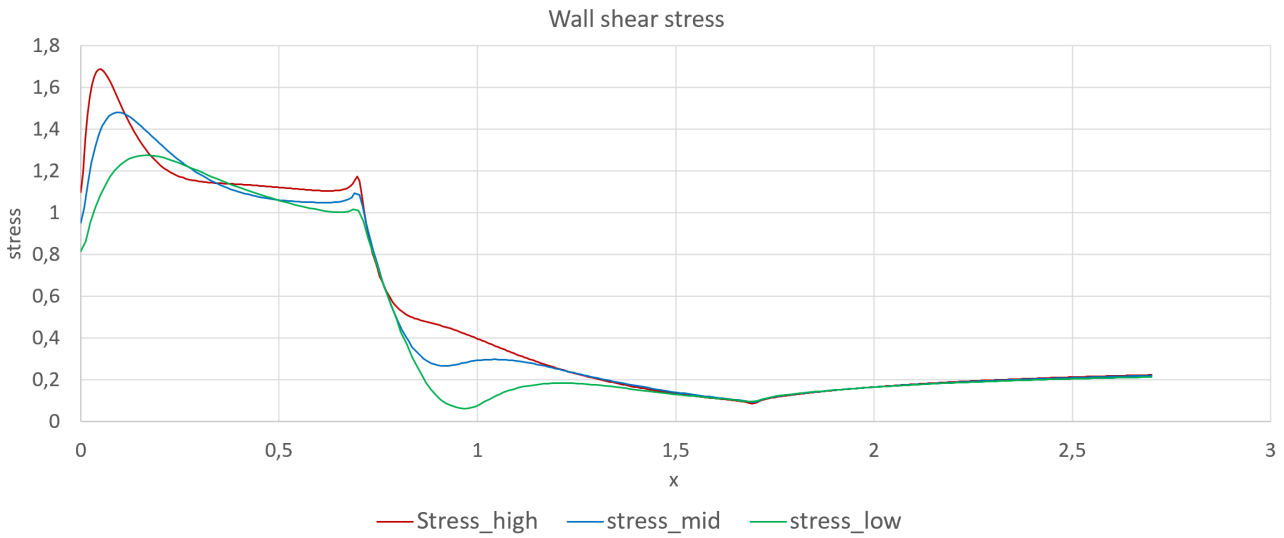


Figure 1.13: Shear stress distribution along the wall channel

The wall pressure profile is, as expected, very similar to the cross-sectional averaged static pressure distribution, (Fig.1.10). It is interesting to note a small pressure peak at the beginning of the divergent section. This behavior is caused by the wall curvature, which locally accelerates the flow. The significantly different behaviour observed for the low mesh confirms that its level of refinement is not sufficient to accurately analyze these local flow features.

Regarding the wall shear stress distribution, a sharp peak is observed near the inlet. This reflects the development of the boundary layer as the flow enters the channel. The low mesh underestimates this peak due to its insufficient near-wall resolution. A pronounced decrease in wall shear stress is then observed at the beginning of the divergent section, corresponding to the deceleration of the flow. The final increase is a return to balanced boundary layer conditions.

## 1.4. Simulation resolving the flow up to the wall

### 1.4.1. Mesh Generation

In this third configuration, a wall-resolved approach is adopted instead of wall functions. So it is necessary a significantly higher grid resolution in the near-wall regions. The resulting computational mesh employed in this simulation is illustrated in Fig. 1.14.

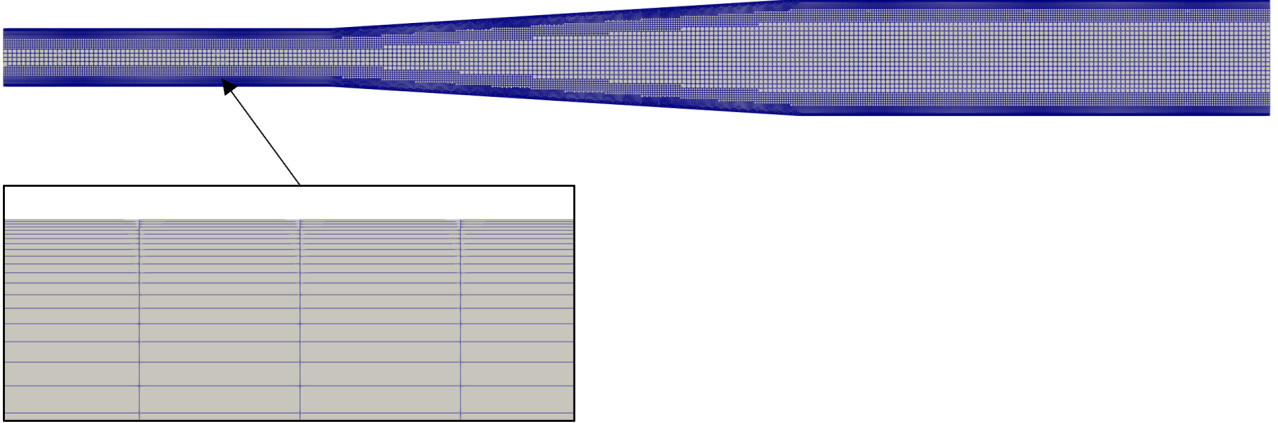


Figure 1.14: Divergent Channel scheme for wall-resolved approach

After the grid generation phase, the mesh quality is rigorously verified using OpenFOAM's *checkMesh* utility. The critical geometric and topological mesh quality metrics are summarized in the subsequent Table 1.5:

|                | N. of cells | Max aspect ratio | Max skewness | Av. non-orthogonality |
|----------------|-------------|------------------|--------------|-----------------------|
| <b>no-wall</b> | 81964       | 80.667           | 0.346969     | 3.32043               |

Table 1.5: Mesh quality metrics

The value of the maximum aspect ratio may appear elevated, but it is attributable to the highly stretched cells generated in the walls boundary layers, so the grid quality remains fully acceptable. Indeed, secondary quality metrics, including skewness and non-orthogonality, fall well within standard recommended values.

The post-simulation non-dimensional wall distance ( $y^+$ ) distributions, computed for the two different models applied to the mesh, are summarized in Table 1.6:

|                             | Min $y^+$ | Average $y^+$ | Max $y^+$ |
|-----------------------------|-----------|---------------|-----------|
| <b><i>k-w</i> SST model</b> | 0.273     | 0.651         | 2.719     |
| <b>SA model</b>             | 0.181     | 0.615         | 2.708     |

Table 1.6: values of  $y^+$

### 1.4.2. $k$ - $\omega$ SST model

#### Boundary Conditions

The boundary conditions for velocity, pressure, and specific dissipation rate ( $\omega$ ) remain identical to those implemented in the wall-function case (see Fig. 1.5 and Fig. 1.6).

This consistency is due to the formulation of the turbulence model, which automatically adapts its near-wall treatment based on the local non-dimensional wall distance ( $y^+$ ). For  $y^+ < 1$ , the grid resolves the viscous sublayer directly, whereas for coarser meshes ( $y^+ > 30$ ), the model switches to wall functions to resolve the log-law region.

Conversely, the boundary conditions that differ from the previous case are detailed below:

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    location     "0";
    object      k;
}
// *****

dimensions      [0 2 -2 0 0 0 0];

inletk          0.81;

internalField    uniform $inletk;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform $inletk;
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      fixedValue;
        value      uniform 0;
    }

    divergent_bottomWall
    {
        type      fixedValue;
        value      uniform 0;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

(a) Turbulent kinetic energy boundary condition

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    location     "0";
    object      nut;
}
// *****

dimensions      [0 2 -1 0 0 0 0];

internalField    uniform 0.012323;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.012323;
    }

    outlet
    {
        type      zeroGradient;
        value      uniform 0.012323;
    }

    divergent_topWall
    {
        type      nutLowReWallFunction;
        value      uniform 0;
    }

    divergent_bottomWall
    {
        type      nutLowReWallFunction;
        value      uniform 0;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

(b) Turbulent Kinematic Viscosity boundary condition

Figure 1.15: Boundary condition for  $k$ - $\omega$  SST model resolving the flow up to the wall

As observed regarding the boundary conditions for the turbulent kinetic energy ( $k$ ), the wall functions along the top and bottom walls have been omitted. Instead, they have been replaced by a *fixedValue* condition set to zero. This parameterization is required to directly resolve the viscous sublayer, enforcing the physical constraint of zero turbulent kinetic energy at the wall surface.

Regarding the turbulent kinematic viscosity ( $\nu_t$ ), the *nutLowReWallFunction* boundary condition has been applied to the solid walls. This formulation acts as a versatile, automated wall treatment designed for low-Reynolds-number grids ( $y^+ < 1$ ), while maintaining compatibility with the overall turbulence architecture. Since the mesh directly resolves the viscous sublayer,

the turbulent viscosity near the wall is physically negligible ( $\nu_t \rightarrow 0$ ). The *nutLowReWallFunction* mathematically enforces this behaviour by calculating  $\nu_t$  directly from the resolved velocity profile and the near-wall cell distance, ensuring a seamless transition and preventing numerical errors in the computation of the wall shear stress.

## Discretization Scheme and Solver

Regarding the numerical discretization schemes, the fvSchemes configuration remains unchanged compared to the case utilizing wall functions.

The same applies to the fvSolution configuration, with the sole exception of the values selected for the under-relaxation factors. Indeed, as illustrated in Fig. 1.16, slightly lower under-relaxation values were chosen for both  $k$  and  $\omega$ .

This adjustment was necessary to enhance numerical stability and ensure smooth convergence. When transitioning from a wall-function approach to a near-wall resolving mesh ( $y^+ \approx 1$ ), the high grid resolution and steep gradients within the viscous sublayer significantly increase the non-linearity of the governing equations, making the simulation more prone to numerical oscillations.

```
relaxationFactors
{
    fields
    {
        p            0.3;
    }
    equations
    {
        U            0.7;
        omega        0.5;
        k            0.5;
    }
}
```

Figure 1.16: Relaxation factors inside fvSolution

### 1.4.3. SA model

#### Boundary Conditions

The boundary conditions for velocity and pressure remain identical to those implemented in the wall-function case (see Fig. 1.5).

The boundary conditions for  $\nu_t$  and  $\tilde{\nu}$ , which characterize the Spalart-Allmaras (SA) turbulence model, are illustrated in the following figure.

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    location    "0";
    object      nut;
}
// ..... //

dimensions      [0 2 -1 0 0 0 0];

internalField   uniform 0.012323;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.012323;
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      calculated;
        value      uniform 0;
    }

    divergent_bottomWall
    {
        type      calculated;
        value      uniform 0;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

(a) Turbulent Kinematic Viscosity boundary condition

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    object      nuTilda;
}
// ..... //

dimensions      [0 2 -1 0 0 0 0];

internalField   uniform 0.012323;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 0.012323;
    }

    outlet
    {
        type      zeroGradient;
    }

    divergent_topWall
    {
        type      fixedValue;
        value      uniform 0;
    }

    divergent_bottomWall
    {
        type      fixedValue;
        value      uniform 0;
    }

    front
    {
        type      empty;
    }

    back
    {
        type      empty;
    }
}
```

(b) Modified Turbulent Kinematic Viscosity boundary condition

Figure 1.17: Boundary condition for SA model resolving the flow up to the wall

A *fixedValue* condition was enforced to prescribe the numerical value of  $\tilde{\nu}$  derived from the following analytical formulation:

$$\tilde{\nu} = C_\mu \frac{k^2}{\epsilon}$$

Instead, the turbulent kinematic viscosity ( $\nu_t$ ) is governed by a *calculated* boundary condition. In the context of the Spalart-Allmaras model,  $\nu_t$  is not directly resolved via an independent transport equation; instead, it is a dependent variable algebraically linked to the modified turbulent viscosity ( $\tilde{\nu}$ ).

## Discretization Scheme and Solver

Regarding the numerical discretization schemes, Fig. 1.18a illustrates the specific methods adopted to approximate the spatial and temporal operators within the governing equations:

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "system";
    object      fvSchemes;
}
// *****

ddtSchemes
{
    default     steadyState;
}

gradSchemes
{
    default     Gauss linear;
}

divSchemes
{
    default     none;
    div(phi,U) bounded Gauss linearUpwind grad(U);
    div(phi,nuTilda) bounded Gauss linearUpwind grad(nuTilda);
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default     Gauss linear corrected;
}

interpolationSchemes
{
    default     linear;
}

snGradSchemes
{
    default     corrected;
}

wallDist
{
    method meshWave;
}
```

(a) Discretization schemes

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "system";
    object      fvSolution;
}
// *****

solvers
{
    p
    {
        solver      GAMG;
        tolerance   1e-12;
        relTol      0;
        smoother     GaussSeidel;
    }

    U
    {
        solver      smoothSolver;
        smoother     GaussSeidel;
        tolerance   1e-10;
        relTol      0;
    }

    nuTilda
    {
        solver      smoothSolver;
        smoother     GaussSeidel;
        tolerance   1e-08;
        relTol      0;
    }
}

SIMPLE
{
    nNonOrthogonalCorrectors 0;

    residualControl
    {
        p          1e-5;
        U          1e-5;
        nuTilda    1e-5;
    }
}

relaxationFactors
{
    fields
    {
        p          0.3;
    }
    equations
    {
        U          0.7;
        nuTilda    0.4;
    }
}
```

(b) Solvers

Figure 1.18: Discretization scheme and solver for SA simulation

A second-order accurate *bounded Gauss linearUpwind grad(nuTilda)* scheme was selected for the convective transport term of the modified turbulent viscosity ( $\tilde{\nu}$ ).

The linear solvers, tolerance criteria, and algorithms selected for the pressure and velocity fields, as well as the turbulence quantities, are detailed in Fig. 1.18b.

#### 1.4.4. Results

The residual for the SST and SA simulations are shown in Fig. 1.19. The two cases exhibit different convergence behaviours. The SST model converges smoothly with all residuals reaching the target value of  $1e-5$  within 2500 iterations. The SA model instead displays visible oscillations in the early stage before settling into a steadier decreasing trend, likely due to the sensitivity of the one-equation model to the adverse pressure gradient region. Although not all residuals reach the target value, monitoring of the quantities values at each timestep confirms that the solution is effectively converged.

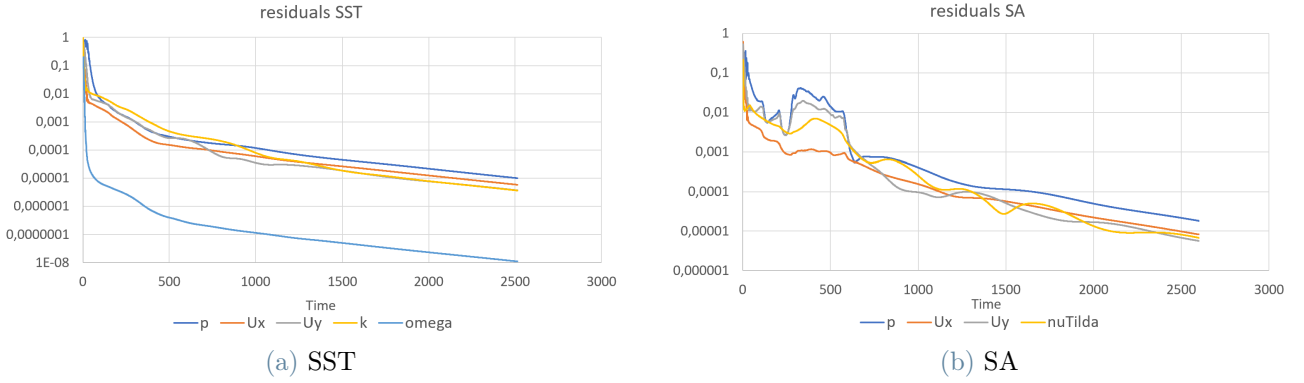


Figure 1.19: Residual for the SST and SA models.

The velocity reduction coefficients are the following:

|            | flow rate, [ $m^3/s$ ] | $\phi$ |
|------------|------------------------|--------|
| <b>SST</b> | 0.247                  | 0.842  |
| <b>SA</b>  | 0.245                  | 0.836  |

Table 1.7: velocity reduction coefficients

The following pages present the distributions of velocity, static pressure, total pressure, wall pressure, and wall shear stress.

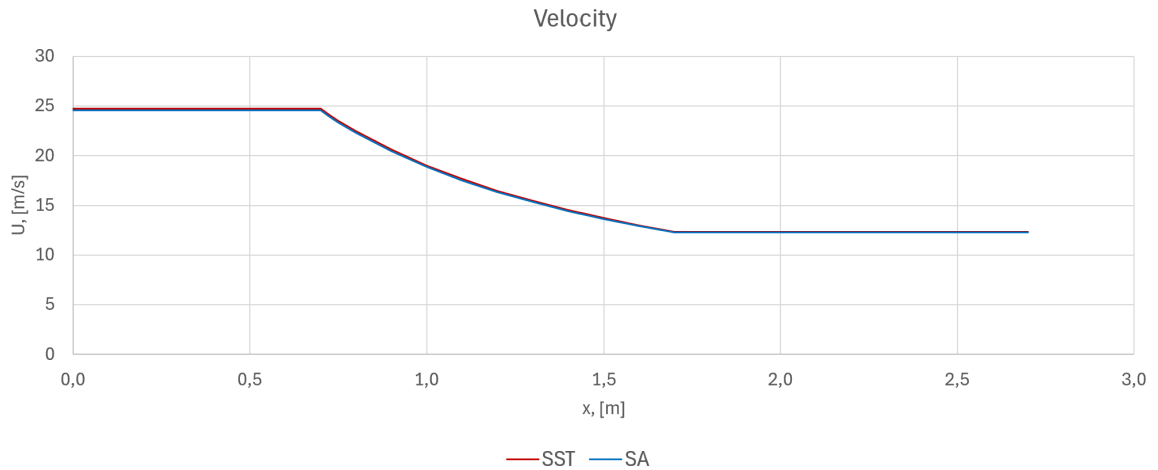


Figure 1.20: Cross-sectional averaged velocity profile

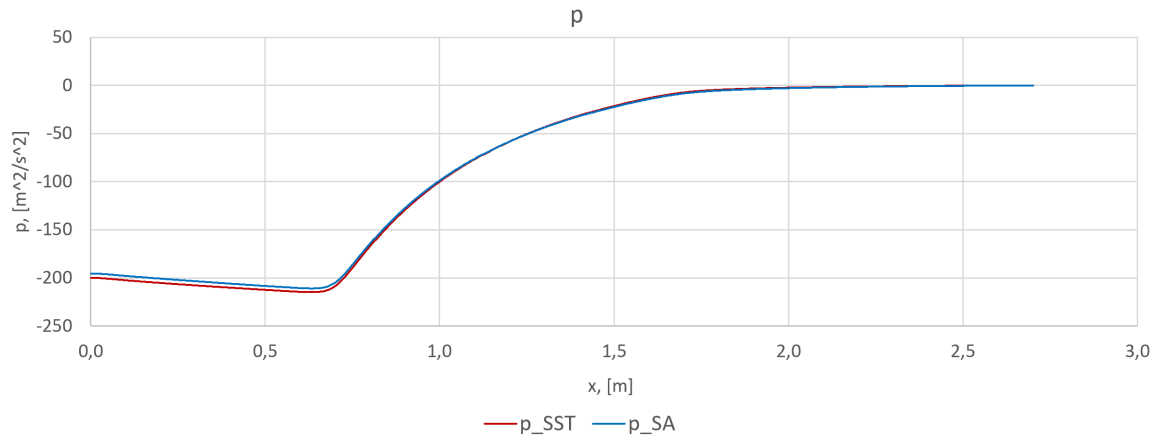


Figure 1.21: Static pressure distribution along the channel

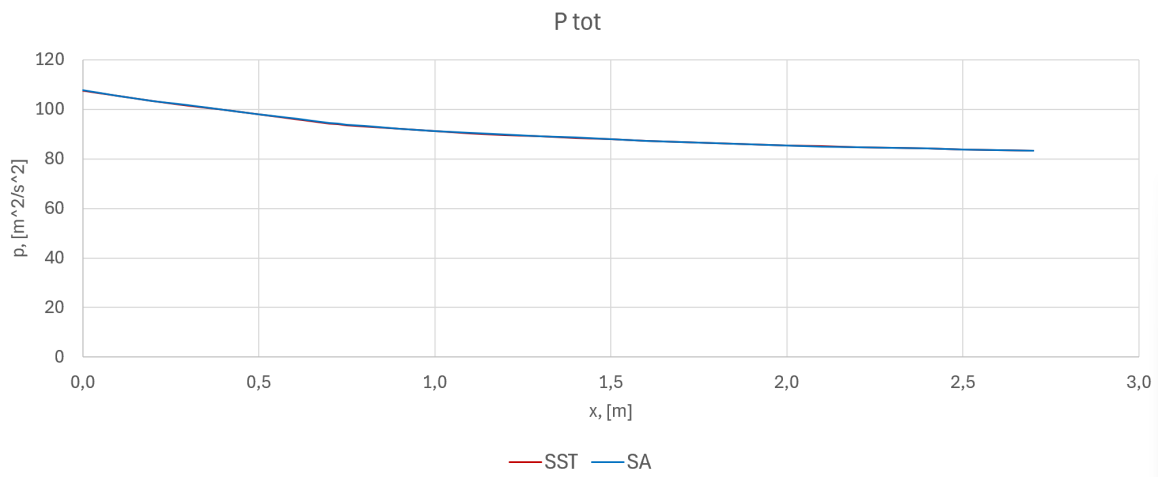


Figure 1.22: Total pressure distribution along the channel

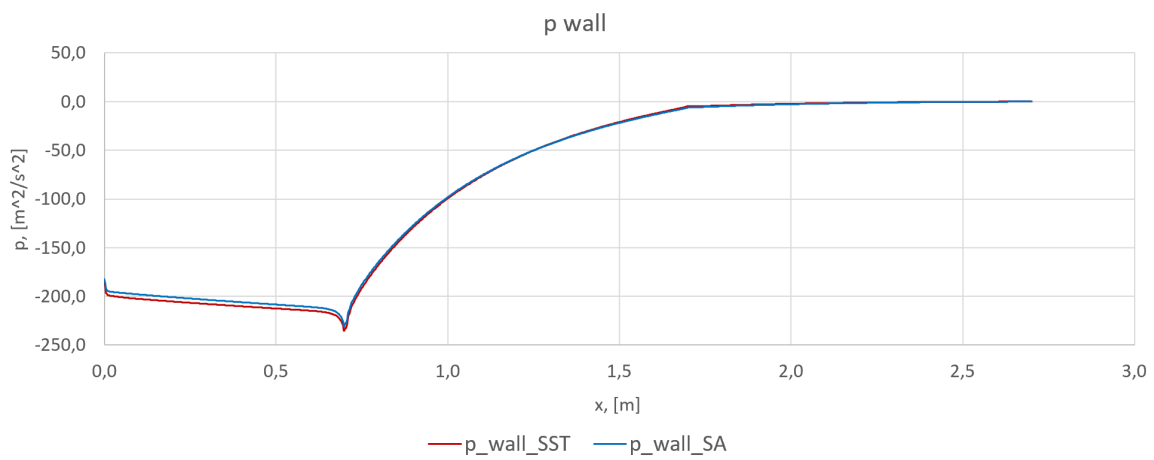


Figure 1.23: Pressure distribution along the wall channel

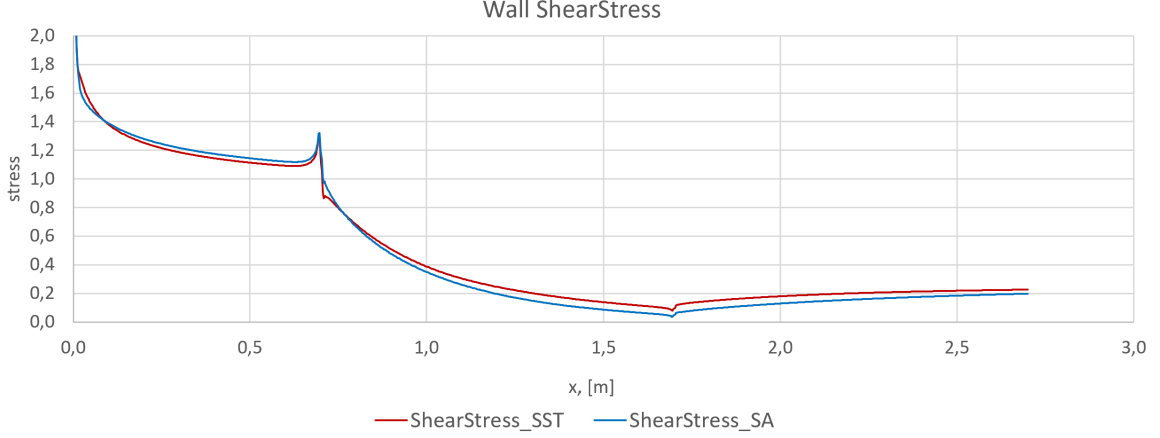


Figure 1.24: Shear stress distribution along the wall channel

The profiles obtained using the SST turbulence model are in excellent agreement with those obtained using the SA turbulence model. Furthermore, the results closely match those obtained in the simulations performed with wall functions, particularly those corresponding to the finest mesh.

This confirms both the convergence of the numerical solutions and the accuracy of the simulations.

## 1.5. Conclusions

The objective of this first part of the report is to analyze the turbulent flow through a two-dimensional diffuser and to analyze the influence of mesh refinement and turbulence modeling on the numerical results.

A grid independence study was first performed using three progressively refined meshes. The comparison of the KPIs showed a very good agreement between the medium and high meshes, demonstrating that grid-independent results had been achieved. The low mesh, on the other hand, exhibited noticeable deviations in several quantities, particularly in the wall pressure and wall shear stress distributions, highlighting the importance of adequate mesh refinement.

The flow behaviour observed in the simulations was consistent with the expected physics. Due to the increase in channel height, the mean velocity decreased along the diffuser, while the static pressure increased as a consequence of pressure recovery. The velocity reduction coefficient remained below unity, indicating that the diffuser did not achieve ideal isentropic performance because of viscous losses and boundary layer effects. The total pressure continuously decreased along the channel as a result of frictional dissipation.

Overall, the comparison between the different meshes, turbulence models, and wall treatments demonstrated that the numerical results are both converged and reliable. The excellent agreement between the simulations with wall-function and the simulation without them provides strong confidence in the accuracy of the simulations and in the physical interpretation of the diffuser flow.

# 2 | Simulation of the 2D incompressible flow around the NACA4415 airfoil

## 2.1. Introduction

The present work concerns the numerical simulation of the two-dimensional incompressible flow around a NACA4415 airfoil, carried out using the open-source CFD solver OpenFOAM. Two turbulence closure models are considered: the *Spalart–Allmaras* (SA) model and the standard  $k-\varepsilon$  model with wall functions. The angles of attack considered are  $5^\circ$  and  $0^\circ$ ; since both correspond to attached or mildly separated flow conditions, a steady-state formulation is adopted throughout.

The study includes a grid independence study conducted on three progressively refined meshes for both turbulence models, in order to assess the sensitivity of the solution to spatial discretization. Additional simulations are performed using the SA model to investigate the effect of near-wall resolution and angle of attack.

The key performance indicators (KPIs) evaluated across all cases are:

1. the lift and drag coefficients ( $C_l$ ,  $C_d$ );
2. the pressure and shear stress distributions along the airfoil surface;
3. the velocity profile in the wake one chord length downstream of the trailing edge.

## 2.2. Geometrical Domain

The computational domain is structured around the NACA4415 airfoil, with chord length  $c = 0.1$  m. The domain extends 5 chord lengths in every direction. The velocity analysis (Figure 2.16) confirmed that the freestream velocity was correctly recovered away from the airfoil surfaces.

A dedicated Python script was used to generate the wall-normal direction at each surface point to define the surrounding layer boundaries.

The mesh adopts a C-grid topology and is divided into two concentric regions: an inner region forming the inflation layer around the airfoil, and an outer region extending to the far-field boundaries. Together, these are organized into 16 structured hexahedral blocks. The airfoil surface and the inflation layer boundary are defined using spline edges, while the outer far-field boundary follows a curved C-shaped arc.

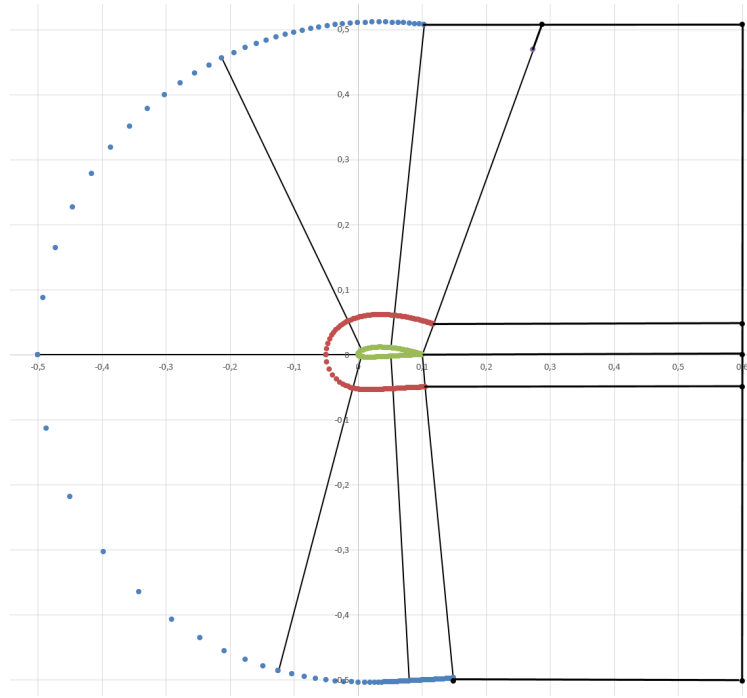


Figure 2.1: blocks

## 2.3. Simulation with wall functions

### 2.3.1. Mesh Generation and Grid Independence Study

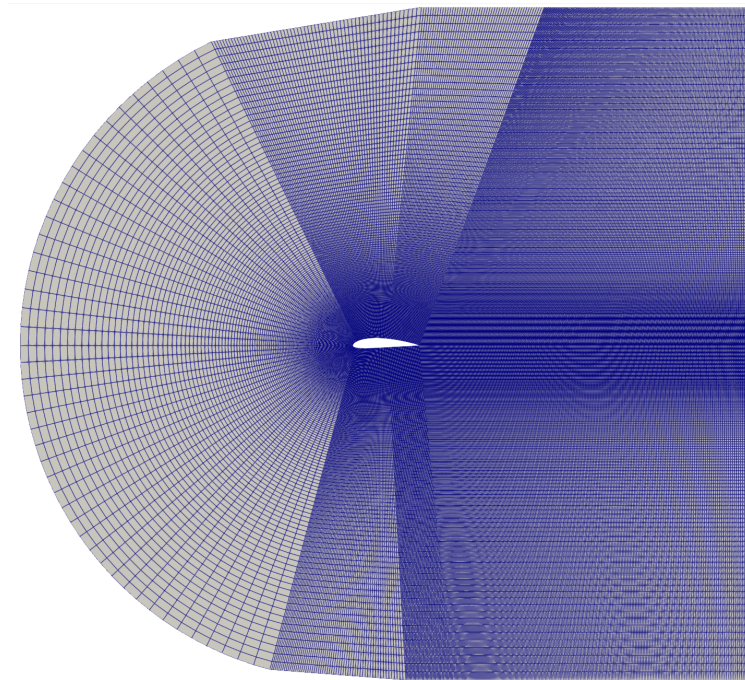


Figure 2.2: Mesh

In order to perform the grid independence study, three meshes of increasing refinement were generated, using the same grid for both the SA and the  $k-\epsilon$  turbulence models. The meshes were built by progressively increasing the number of cells and the gradient along the wall-normal direction, without following a fixed refinement ratio between successive levels.

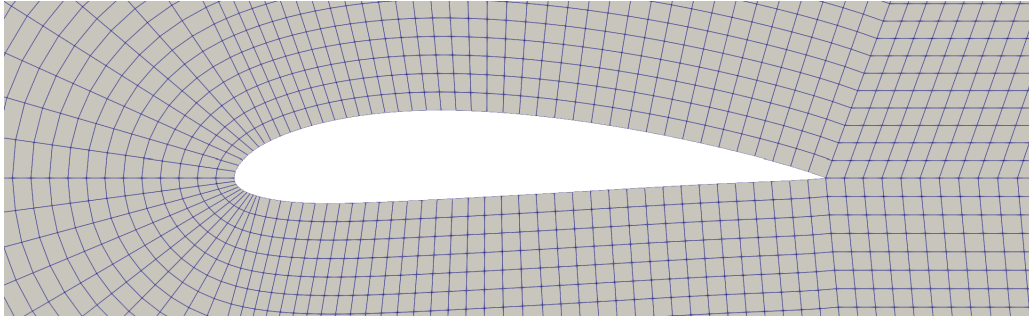


Figure 2.3: Mesh 1: low refinement

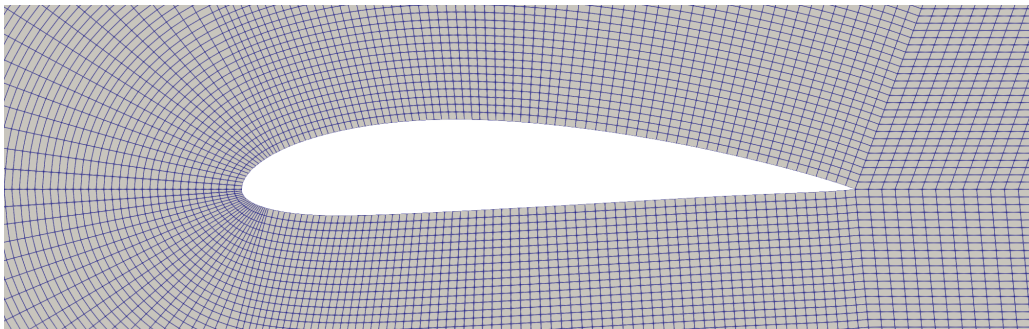


Figure 2.4: Mesh 2: mid refinement

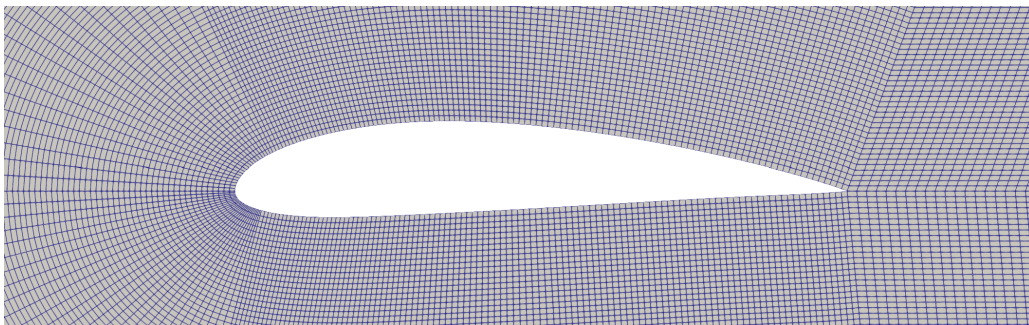


Figure 2.5: Mesh 3: high refinement

Looking at the quality metrics reported in Table 2.1, it can be concluded that the meshes are of satisfactory quality for the simulations. In particular, the progressive decrease in the *average non-orthogonality* indicates that the mesh quality improves consistently with refinement.

|             | N. of cells | Max aspect ratio | Max skewness | Av. non-orthogonality |
|-------------|-------------|------------------|--------------|-----------------------|
| <b>low</b>  | 20460       | 4.890            | 1.174        | 8.418                 |
| <b>mid</b>  | 54560       | 6.708            | 1.181        | 8.103                 |
| <b>high</b> | 109430      | 6.513            | 1.185        | 8.030                 |

Table 2.1: Mesh quality metrics

Since wall functions were adopted for both turbulence models, the mesh refinement was limited by the requirement of maintaining an average  $y^+ > 30$ . This criterion determined the maximum level of refinement used in the grid independence study.

The reported  $y^+$  values (Table 2.2) are average values over the airfoil surface. Although some local regions exhibit  $y^+ < 30$ , this is acceptable because these isolated values occur in low-velocity areas, where  $y^+$  naturally decreases. Therefore, the average  $y^+$  is considered the most relevant parameter for assessing the suitability of the mesh.

Table 2.2: Mesh characteristics and  $y^+$  values for the three refinement levels

| Mesh   | $y^+$ (SA) | $y^+$ (k- $\epsilon$ ) |
|--------|------------|------------------------|
| Low    | 104        | 88                     |
| Medium | 48         | 54                     |
| High   | 36         | 51                     |

### 2.3.2. SA model

#### Boundary Conditions

As shown in Figure 2.6a, the angle of attack of  $5^\circ$  is considered by decomposing the inlet velocity into its Cartesian components, rather than rotating the geometry. The resulting components are  $U_x = 20 \cos(5^\circ) \approx 19.92$  m/s and  $U_y = 20 \sin(5^\circ) \approx 1.74$  m/s. A *fixedValue* condition is applied at the inlet, while *zeroGradient* is used at the outlet, assuming a fully developed flow at the exit section. The airfoil surface is treated as a no-slip wall ( $U = 0$ ), which is the appropriate physical condition for a viscous flow over a solid boundary. The lateral boundaries (*sy* and *dx*) are set to *empty*, as required by OpenFOAM for two-dimensional simulations. The top and bottom boundaries are assigned the *freestreamVelocity* condition: since the angle of attack is non-zero, the flow is not aligned with the domain boundaries and these patches may act locally as either inlet or outlet depending on the flow direction. The *freestream* condition handles both cases consistently, imposing the far-field velocity when the flow enters and reverting to *zeroGradient* when it exits.

As shown in Figure 2.6b, the pressure field uses the kinematic pressure  $p/\rho$  (dimensions  $\text{m}^2/\text{s}^2$ ), as it's standard for incompressible solvers in OpenFOAM. A *fixedValue* of zero is imposed at the outlet, setting the pressure reference level at the exit; a *zeroGradient* condition is applied at both the inlet and the airfoil surface. The top and bottom boundaries use *freestreamPressure*, coherently with the velocity boundary condition applied on the same patches. Since the incompressible solver operates on relative pressure, the absolute pressure level has no influence on the computed flow field, as only pressure gradients appear in the momentum equations. Setting the outlet to 1 bar was avoided as it significantly slows down convergence: the solver must resolve small pressure variations on top of a large absolute value, degrading the numerical conditioning of the linear system. The outlet is therefore set to zero, and a uniform offset of 1 bar ( $83333.333 \text{ m}^2/\text{s}^2$ ) has been added to all pressure plots for consistency with the specified ambient conditions.

Regarding the turbulent eddy viscosity  $\nu_T$ , shown in Figure 2.6c, the inlet value is computed from the prescribed turbulence intensity and length scale and assigned via a *calculated* type, while *zeroGradient* is used at the outlet. On the airfoil surface, the *nutUSpaldingWallFunction* is employed. The top and bottom boundaries are set to *freestream* consistently with the other fields.

```

FoamFile
{
    format      ascii;
    class       volVectorField;
    object      U;
}
// *****

dimensions      [0 1 -1 0 0 0];

internalField    uniform (19.92389 1.74311 0);

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform (19.92389 1.74311 0);
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      noSlip;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      freestreamVelocity;
        freestreamValue uniform (19.92389 1.74311 0);
    }

    bottom
    {
        type      freestreamVelocity;
        freestreamValue uniform (19.92389 1.74311 0);
    }
}

```

(a) Velocity boundary condition

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    object      nut;
}
// *****

dimensions      [0 2 -1 0 0 0];

internalField    uniform 0.00134164;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.00134164;
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      nutUSpaldingWallFunction;
        value      uniform 0;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }

    bottom
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }
}

```

(c) Turbulent Kinematic Viscosity boundary condition

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    object      p;
}
// *****

dimensions      [0 2 -2 0 0 0];

internalField    uniform 0;

boundaryField
{
    inlet
    {
        type      zeroGradient;
    }

    outlet
    {
        type      fixedValue;
        value      uniform 0;
    }

    profile
    {
        type      zeroGradient;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      freestreamPressure;
        freestreamValue uniform 0;
    }

    bottom
    {
        type      freestreamPressure;
        freestreamValue uniform 0;
    }
}

```

(b) Pressure boundary condition

```

FoamFile
{
    format      ascii;
    class       volScalarField;
    object      nuTilda;
}
// *****

dimensions      [0 2 -1 0 0 0];

internalField    uniform 0.00134164;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 0.00134164;
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      fixedValue;
        value      uniform 0;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }

    bottom
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }
}

```

(d) Modified Turbulent Kinematic Viscosity boundary condition

Figure 2.6: Boundary condition for SA model with wall function

## Discretization Scheme and Solver

The numerical discretization settings configured in the *fvSchemes* dictionary for the *Spalart-Allmaras* (SA) simulation are presented in Fig. 2.7.

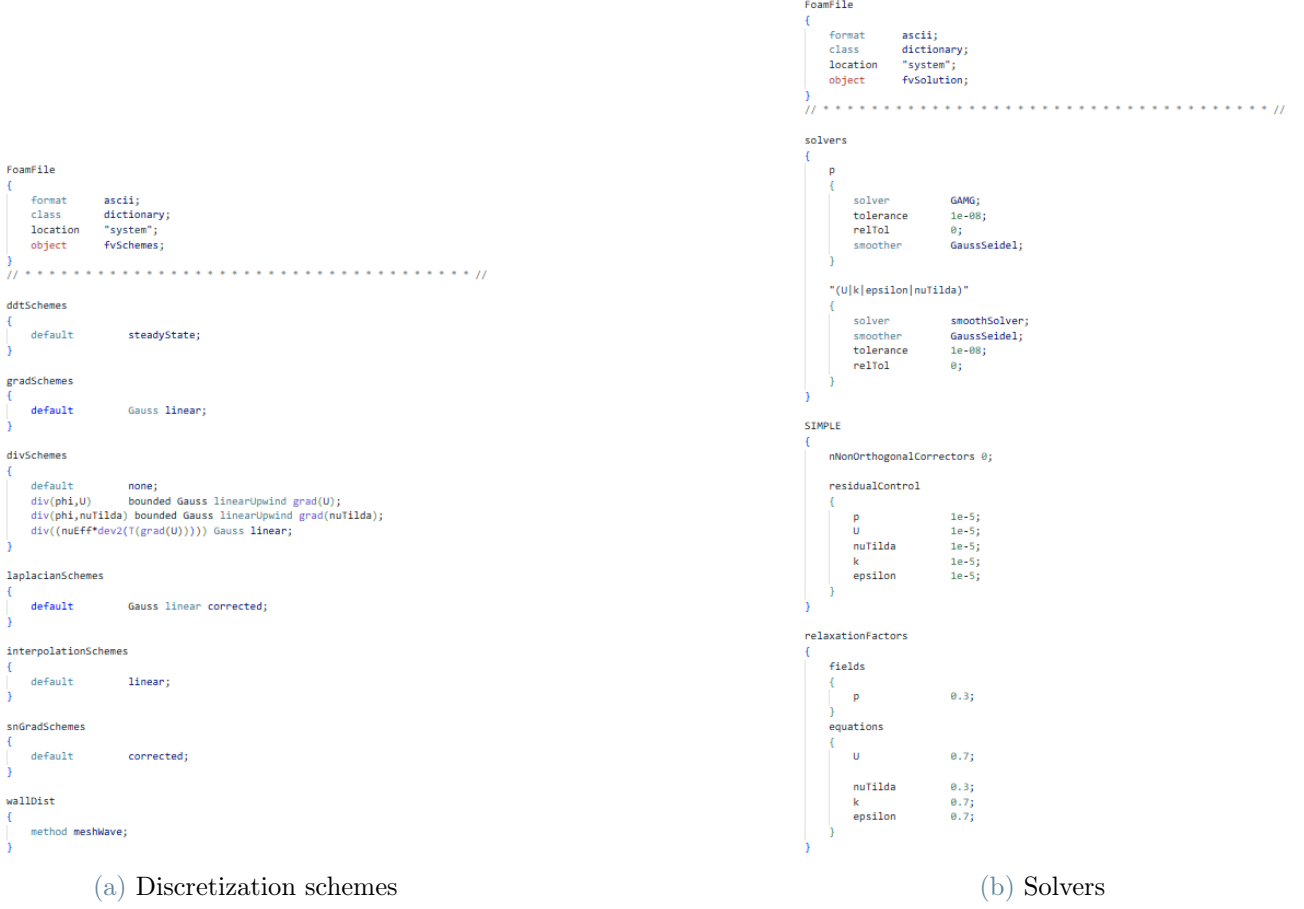


Figure 2.7: Discretization scheme and solver for SA simulation

As illustrated, a second-order accurate *linearUpwind* scheme is systematically applied to the convective transport terms of both the velocity field ( $\mathbf{U}$ ) and the modified turbulent viscosity ( $\tilde{\nu}$ ) to achieve superior spatial resolution and minimize numerical diffusion.

The *fvSolution* dictionary used for the simulations is reported in Fig. 2.7b.

This file defines the numerical solvers and convergence tolerances for the velocity, pressure, and turbulence variables. Thanks to the modular structure of OpenFOAM, the same *fvSolution* file can be used for different turbulence models, while the solver automatically selects only the required equations.

The file also specifies the under-relaxation factors used within the SIMPLE algorithm to ensure a stable convergence. A lower relaxation factor is adopted for the SA turbulence variable because of its stronger non-linearity, while higher values are used for the  $k$ - $\epsilon$  variables to improve the convergence rate without affecting stability.

### 2.3.3. $k-\varepsilon$ model

#### Boundary Conditions

Regarding the boundary conditions for the velocity field ( $\mathbf{U}$ ), static pressure ( $p$ ), and turbulent kinematic viscosity ( $\nu_t$ ), the configuration remains identical to the setup previously implemented for the *Spalart-Allmaras* (SA) model, as detailed in Fig. 2.6.

Instead, the specific boundary conditions required for the turbulent kinetic energy ( $k$ ) and the turbulent dissipation rate ( $\varepsilon$ ) fields within the  $k-\varepsilon$  model framework are presented in the following Fig. 2.8 .

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    object       k;
}
// ..... //

dimensions      [0 2 -2 0 0 0 0];

inletk          0.06;

internalField    uniform $inletk;

boundaryField
{
    profile
    {
        type      kqRWallFunction;
        value      uniform $inletk;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    outlet
    {
        type      zeroGradient;
    }

    inlet
    {
        type      fixedValue;
        value      uniform $inletk;
    }

    top
    {
        type      freestream;
        freestreamValue uniform $inletk;
    }

    bottom
    {
        type      freestream;
        freestreamValue uniform $inletk;
    }
}
```

(a) Turbulent kinetic energy boundary condition

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    object       epsilon;
}
// ..... //

dimensions      [0 2 -3 0 0 0 0];

inletEpsilon     0.24149;

internalField     uniform $inletEpsilon;

boundaryField
{
    profile
    {
        type      epsilonWallFunction;
        value      uniform $inletEpsilon;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    outlet
    {
        type      zeroGradient;
    }

    inlet
    {
        type      fixedValue;
        value      uniform $inletEpsilon;
    }

    top
    {
        type      freestream;
        freestreamValue uniform $inletEpsilon;
    }

    bottom
    {
        type      freestream;
        freestreamValue uniform $inletEpsilon;
    }
}
```

(b) Epsilon boundary condition

Figure 2.8: Boundary condition for  $k-\varepsilon$  model with wall function

As illustrated, the *freeStream* boundary condition has been implemented at both the top and bottom patches, it acts as a robust, non-reflecting open-boundary formulation specifically designed for external aerodynamic simulations.

## Discretization Scheme and Solver

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "system";
    object      fvSchemes;
}

// *****

ddtSchemes
{
    default     steadyState;
}

gradSchemes
{
    default     Gauss linear;
}

divSchemes
{
    default     none;
    div(phi,U)  bounded Gauss linearUpwind grad(U);
    div(phi,k)   bounded Gauss upwind;
    div(phi,epsilon) bounded Gauss upwind;
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default     Gauss linear corrected;
}

interpolationSchemes
{
    default     linear;
}

snGradSchemes
{
    default     corrected;
}

wallDist
{
    method meshWave;
}
```

Figure 2.9: Discretization schemes

As shown in Figure 2.9, the divergence schemes for the transported turbulent quantities  $k$  and  $\varepsilon$  in the  $k$ - $\varepsilon$  model are set to first-order *upwind*, rather than the second-order *linearUpwind* scheme used for the velocity field. This choice was dictated by numerical stability: when *linearUpwind* was applied to  $k$  and  $\varepsilon$ , the simulation consistently diverged within the first iterations, making it impossible to reach a converged solution. Switching to *upwind* for these two quantities resolved the issue while preserving second-order accuracy on the velocity field, which is the primary variable of interest.

The *fvSolution* dictionary remains identical to the configuration previously illustrated in Fig. 2.7.

### 2.3.4. Results

The first aspect to be verified is the correct convergence of the residuals. The residuals of the simulations with the SA model are shown in Fig. 2.10, while the residuals for the simulations with the k- $\varepsilon$  model are shown in Fig. 2.11. All simulations successfully reached the target residual value of  $1e-5$ .

The simulations performed with the k- $\varepsilon$  model converged faster than those using the SA model. This behaviour can be attributed to the fact that the k- $\varepsilon$  model is particularly well suited for simulations using wall functions.

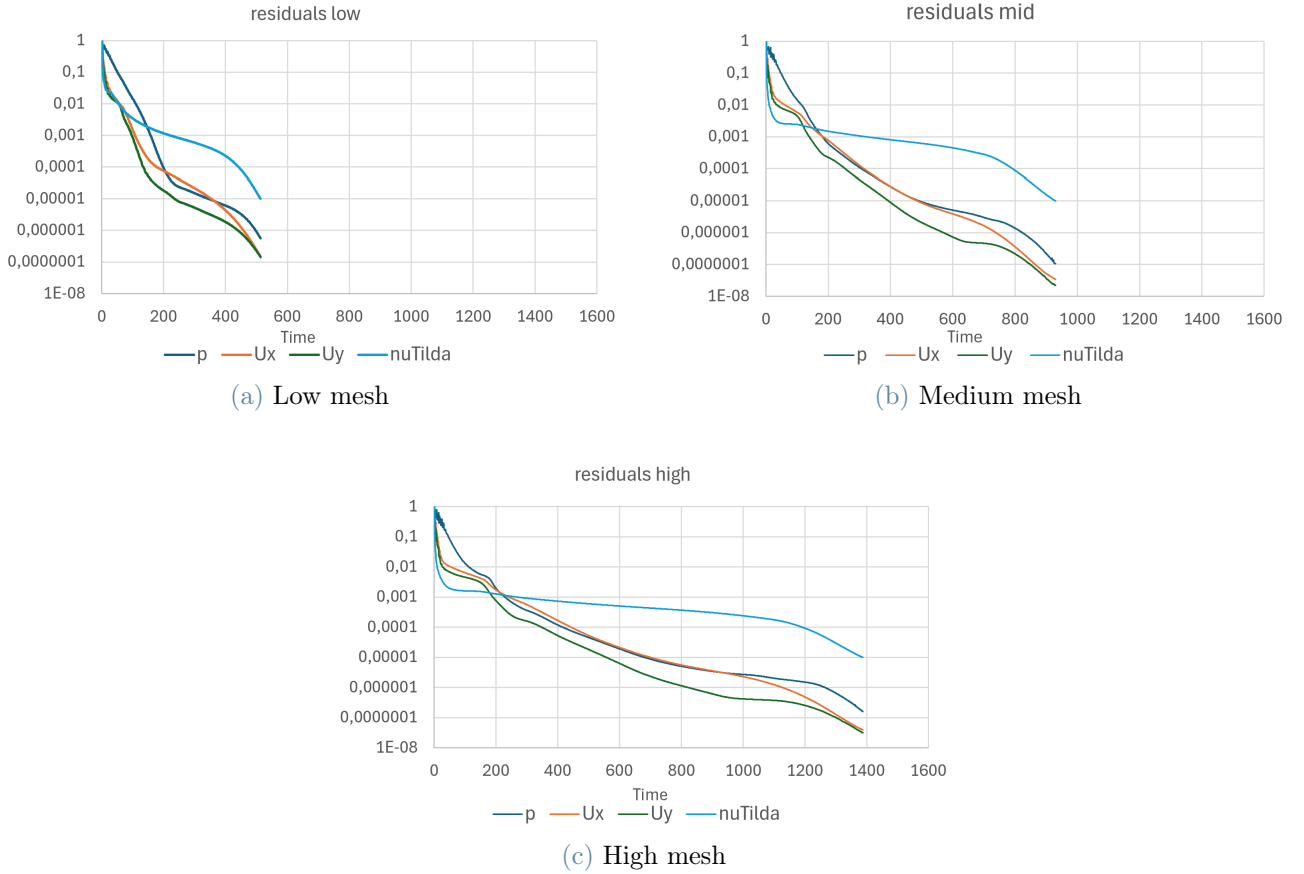


Figure 2.10: Residual for the three mesh resolutions using the SA model.

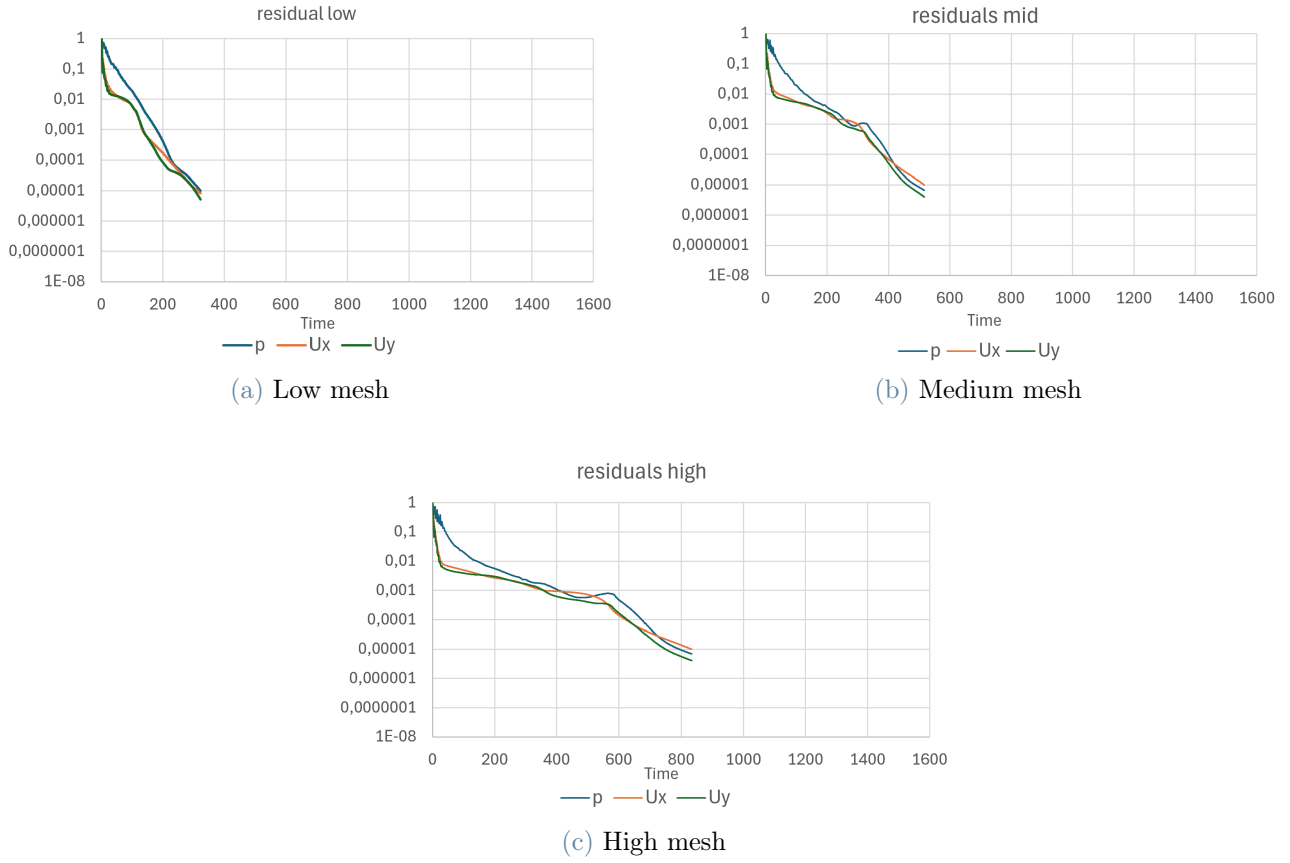


Figure 2.11: Residual for the three mesh resolutions using  $k-\varepsilon$  model.

The lift and drag coefficients obtained with the SA and  $k-\varepsilon$  turbulence models for the three mesh refinement levels are reported in Table 2.3. The very small difference between the results obtained with the medium and fine meshes confirms that grid independence has been achieved. So, the medium mesh can be considered sufficiently refined to provide mesh-independent predictions of both lift and drag coefficients.

| Mesh   | SA    |        | Mesh   | $k-\varepsilon$ |        |
|--------|-------|--------|--------|-----------------|--------|
|        | $C_l$ | $C_d$  |        | $C_l$           | $C_d$  |
| Low    | 0.632 | 0.0496 | Low    | 0.581           | 0.0557 |
| Medium | 0.801 | 0.0355 | Medium | 0.763           | 0.0421 |
| High   | 0.805 | 0.0347 | High   | 0.772           | 0.0422 |

Table 2.3: Lift and drag coefficients obtained with the  $k-\varepsilon$  and SA turbulence models for the three mesh resolutions.

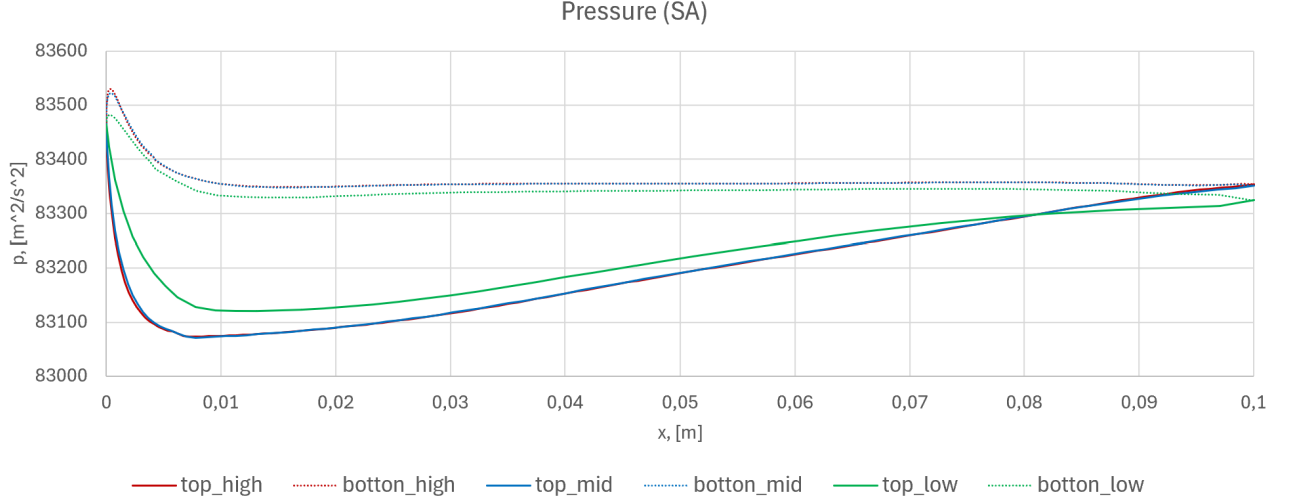


Figure 2.12: Pressure distribution on the profile walls for SA model

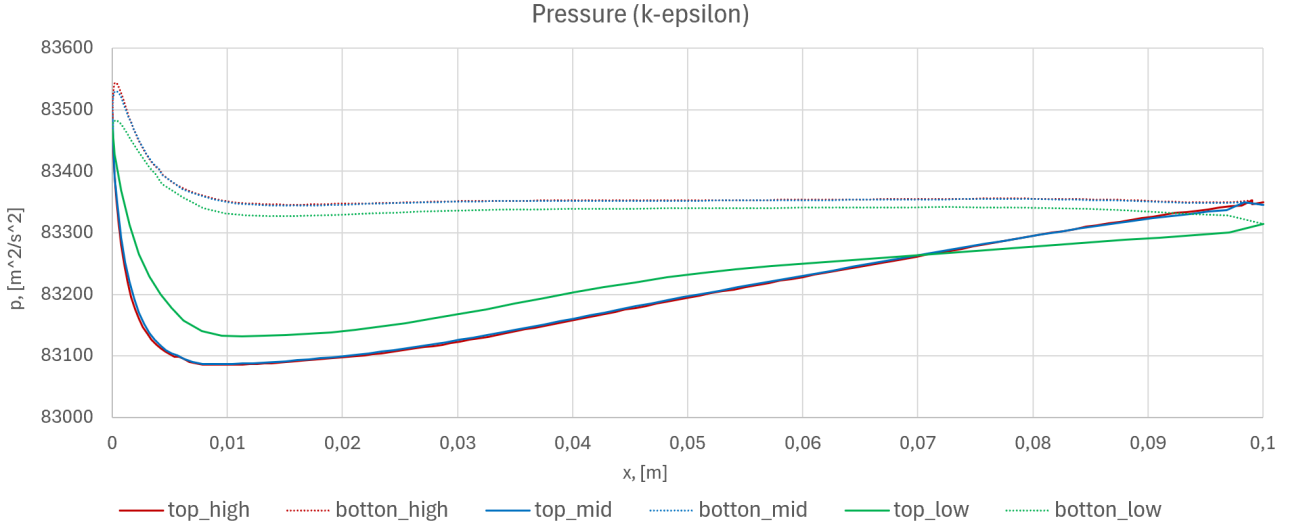


Figure 2.13: Pressure distribution on the profile walls for  $k-\varepsilon$  model

The pressure distribution along the airfoil surfaces (upper and lower walls) shows an excellent agreement between the medium and high meshes. This further confirms that grid-independent results have been achieved.

The pressure field is consistent with the expected aerodynamic behaviour. In particular, a region of low pressure is observed on the upper surface, while a higher pressure is present on the lower surface. This pressure difference is responsible for the generation of lift.

No significant differences are observed between the SA and  $k-\varepsilon$  turbulence models. The close agreement between the two sets of results confirms the stability and reliability of the numerical simulations.

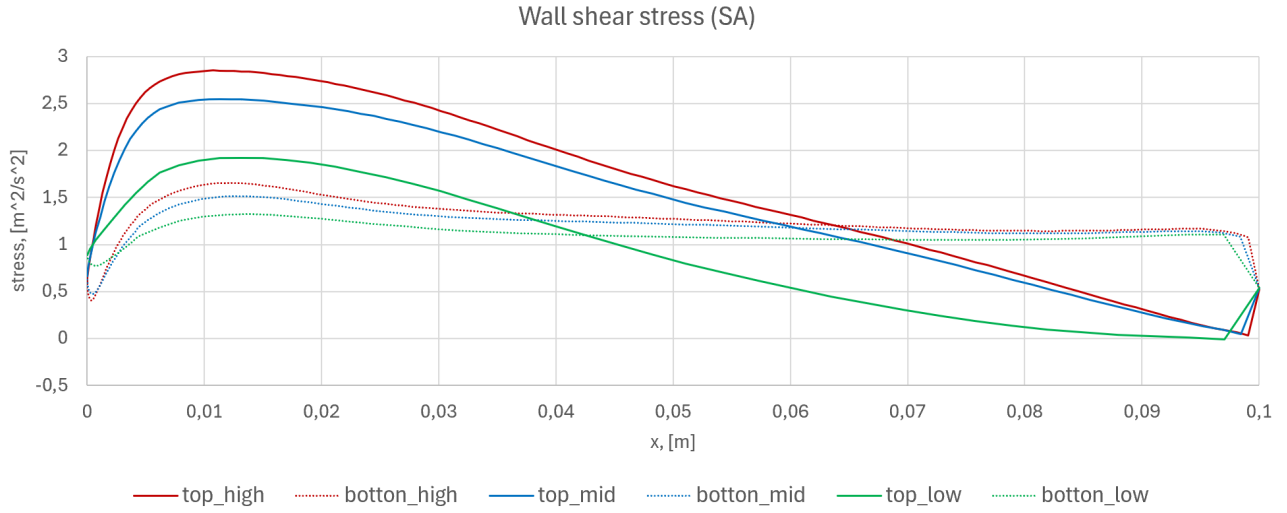


Figure 2.14: Shear stress distribution on the profile walls for SA model

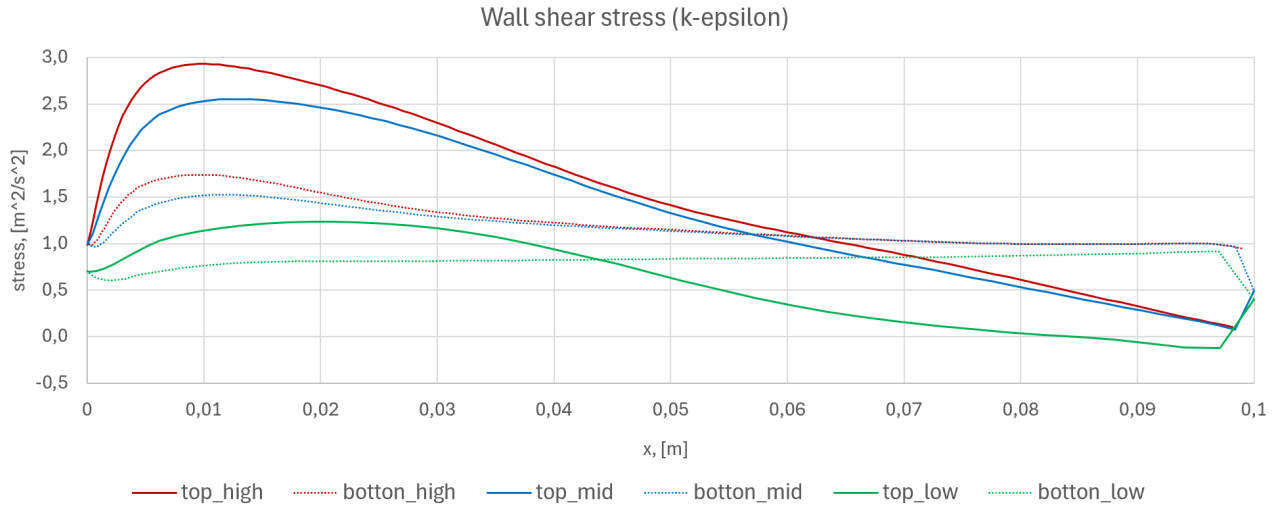


Figure 2.15: Shear stress distribution on the profile walls for k- $\epsilon$  model

The wall shear stress distribution along the airfoil surfaces shows very similar results for the two finest meshes. Both cases exhibit a pronounced peak near the leading edge, followed by a gradual decrease along the airfoil surface.

No significant differences are observed between the two turbulence models.

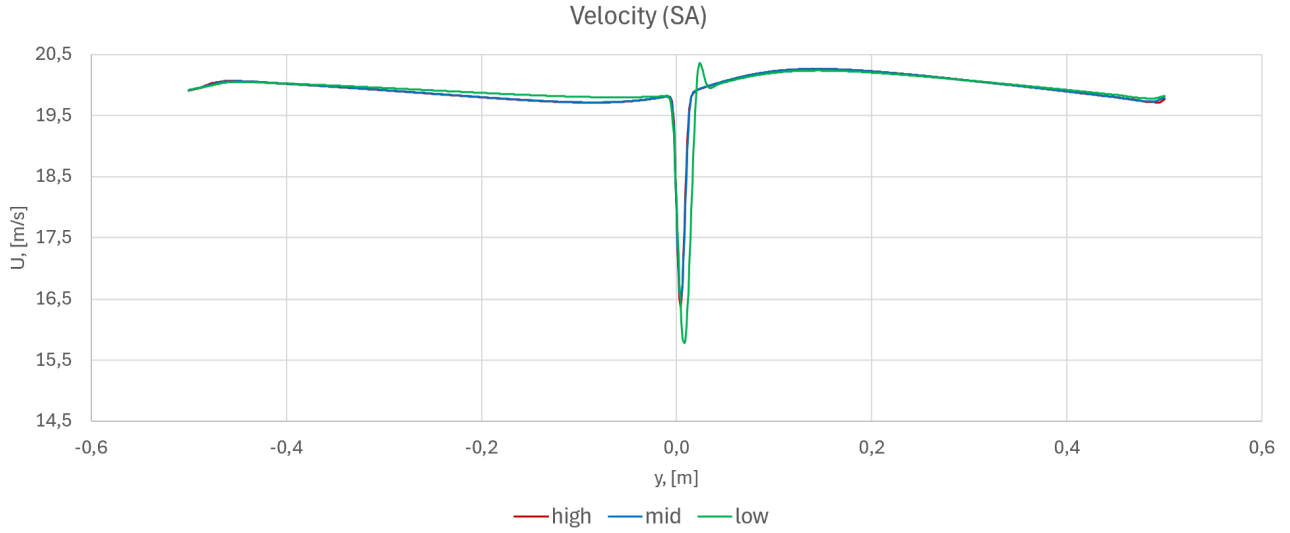


Figure 2.16: Velocity distribution in the wake of the airfoil, one chord downstream of TE for the SA model

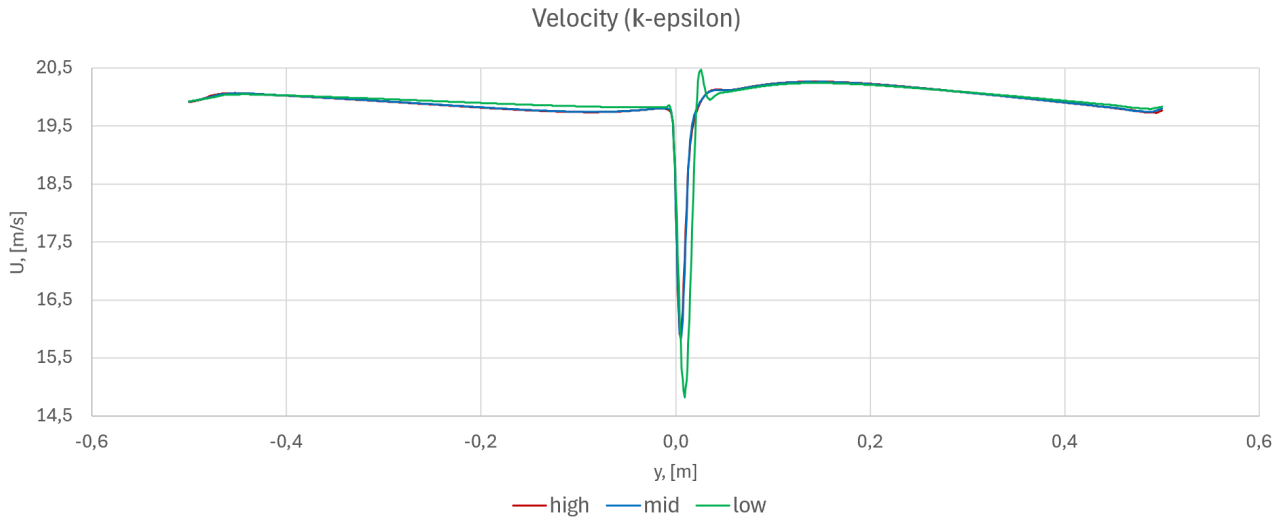


Figure 2.17: Velocity distribution in the wake of the airfoil, one chord downstream of TE for the k- $\epsilon$  model

Figures 2.16 and 2.17 show the velocity distribution in the wake of the airfoil, one chord downstream of TE.

Outside the wake region the velocity recovers to the freestream value indicating that the computational domain is sufficiently large to avoid boundary effects.

At the wake centre, a clear velocity reduction is observed, with a minimum of 15.5–16.5 m/s, due to momentum loss in the boundary layers shed from the airfoil surfaces. Here, a slight asymmetry is present, with a small velocity overshoot on the upper side of the wake, consistent with the 5° angle of attack.

## 2.4. Simulation resolving the flow up to the wall

### 2.4.1. Mesh Generation

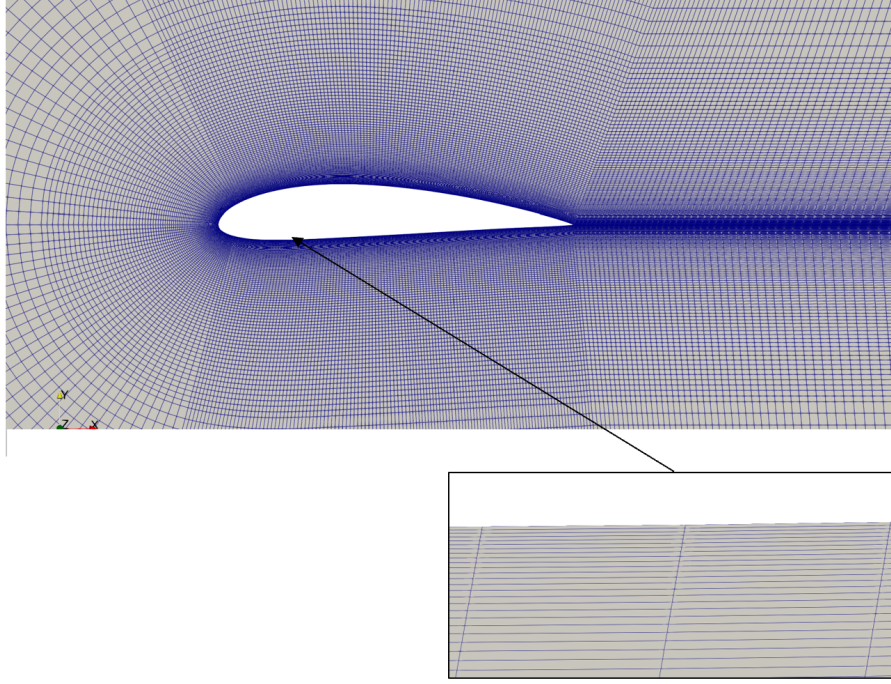


Figure 2.18: Mesh for simulations without wall functions

In order to perform wall-resolved simulations, a mesh with ( $y^+ < 1$ ) was required. The mesh was generated by modifying the *blockMesh* setup, with particular attention to the multi-grading strategy and the smooth transition between adjacent cells.

|     | N. of cells | Max aspect ratio | Max skewness | Av. non-orthogonality |
|-----|-------------|------------------|--------------|-----------------------|
| low | 96800       | 971.297          | 0.921        | 8.337                 |

Table 2.4: Mesh quality metrics

The relatively high value of the *Max aspect ratio* is a consequence of the strong near-wall refinement needed to achieve ( $y^+ < 1$ ). Although such a high aspect ratio is generally not recommended, no numerical issues or deterioration of the solution quality were observed in the simulations.

The values of  $y^+$  obtained for the two simulations are reported in the following table. The small differences between the two cases are due to the slight differences in the flow velocity, which affect the wall shear stress and consequently the local ( $y^+$ ) values.

|       | Min $y^+$ | Average $y^+$ | Max $y^+$ |
|-------|-----------|---------------|-----------|
| SA-5° | 0.053     | 0.515         | 1.145     |
| SA-0° | 0.051     | 0.527         | 1.067     |

Table 2.5: Values of  $y^+$

## 2.4.2. SA model with AOA=5°

### Boundary Conditions

```
FoamFile
{
    format      ascii;
    class       volScalarField;
    object      nut;
}
// *****

dimensions      [0 2 -1 0 0 0 0];

internalField    uniform 0.00134164;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0.00134164;
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      fixedValue;
        value      uniform 0;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }

    bottom
    {
        type      freestream;
        freestreamValue uniform 0.00134164;
    }
}
```

Figure 2.19: Boundary condition for SA model resolving the flow up to the wall

The boundary conditions for the wall-resolved SA simulation are the same as those described in the previous case (Figure 2.6), except for the airfoil surface. Since wall functions are no longer employed,  $\nu_T$  is prescribed using a *fixedValue* boundary condition equal to zero at the wall.

### Discretization Scheme and Solver

Discretization schemes and solvers are identical to the ones used in the case of the Spalart-Allmaras model with wall functions (figure 2.7).

### 2.4.3. SA model with $\text{AOA}=0^\circ$

#### Boundary Conditions

```
FoamFile
{
    format      ascii;
    class       volVectorField;
    object      U;
}
// *****

dimensions     [0 1 -1 0 0 0 0];

internalField   uniform ( 20 0 0);

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform (20 0 0);
    }

    outlet
    {
        type      zeroGradient;
    }

    profile
    {
        type      noSlip;
    }

    sx
    {
        type      empty;
    }

    dx
    {
        type      empty;
    }

    top
    {
        type      symmetry;
    }

    bottom
    {
        type      symmetry;
    }
}
```

Figure 2.20: Boundary condition for SA model resolving the flow up to the wall

The boundary conditions for the SA model at zero angle of attack are very similar to those described in the previous case (Fig. 2.19). The main difference concerns the inlet velocity, which is now aligned with the (x)-axis and imposed as ( $U=(20,0,0)$ ) m/s, with no transverse component.

A second modification involves the top and bottom boundaries. Since the flow at zero angle of attack is symmetric with respect to the horizontal mid-plane of the domain, symmetry boundary conditions can be applied. This choice is consistent with the expected flow physics and helps minimize artificial boundary effects. Only the velocity field is shown in Fig. 2.20; however, the same considerations are applied to all the other quantities.

#### Discretization Scheme and Solver

Discretization schemes and solvers are identical to the ones used in the case of the Spalart-Allmaras model with wall functions (figure 2.7).

#### 2.4.4. Results

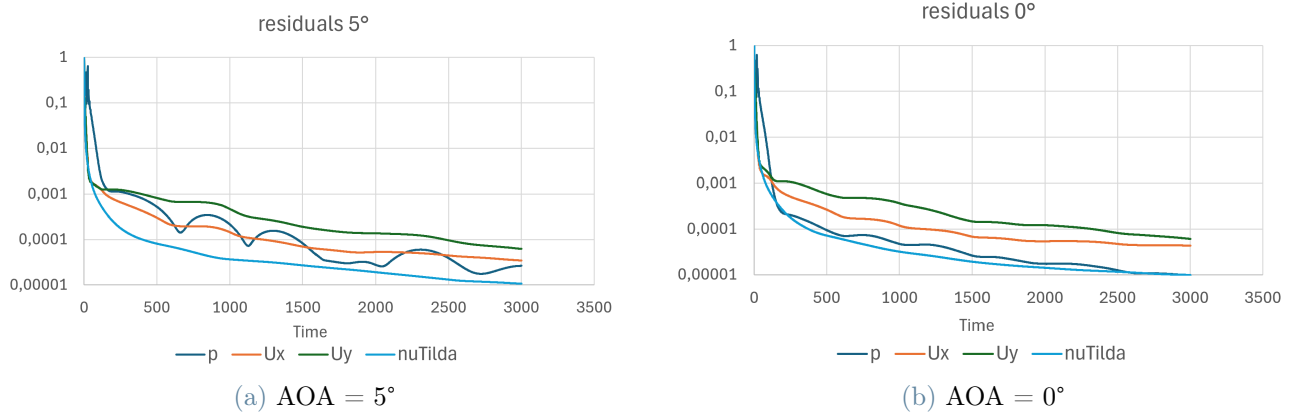


Figure 2.21: Residuals for SA simulation with an AOA = 5° and AOA = 0°.

The residual for the wall-resolving SA simulations at AOA = 5° and AOA = 0° are shown in Figure 2.21. Neither case reaches the 1e-5 target for all residuals within 3000 iterations, with Uy and Ux stalling around 1e-4. However, as for the wall-function cases, convergence of the quantities was verified and confirmed to be achieved well before the end of the run. A notable difference between the two cases is the presence of oscillations in the AOA = 5° simulation, which are absent in the AOA = 0° case. This is physically consistent: at non-zero angle of attack the asymmetric loading generates a more complex flow field around the profile, making the iterative process less stable.

| Mesh     | SA    |        |
|----------|-------|--------|
|          | $C_l$ | $C_d$  |
| AOA = 5° | 0.827 | 0.0335 |
| AOA = 0° | 0.381 | 0.0247 |

Table 2.6: Lift and drag coefficients for the SA turbulence model.

The lift and drag coefficients obtained from the wall-resolved SA simulations are reported in Table 2.6.

It can be observed that the lift coefficient is slightly higher than that obtained from the corresponding simulations using wall functions. This difference is expected because wall functions introduce an approximation of the near-wall flow, particularly in the viscous sublayer. By resolving the boundary layer down to the wall, the wall-resolved simulations are able to provide a more accurate prediction of the aerodynamic forces.

As expected, the case with AOA=5° produces a significantly higher lift coefficient than the case with AOA=0°. This confirms the strong influence of the angle of attack on the aerodynamic loading of the airfoil.

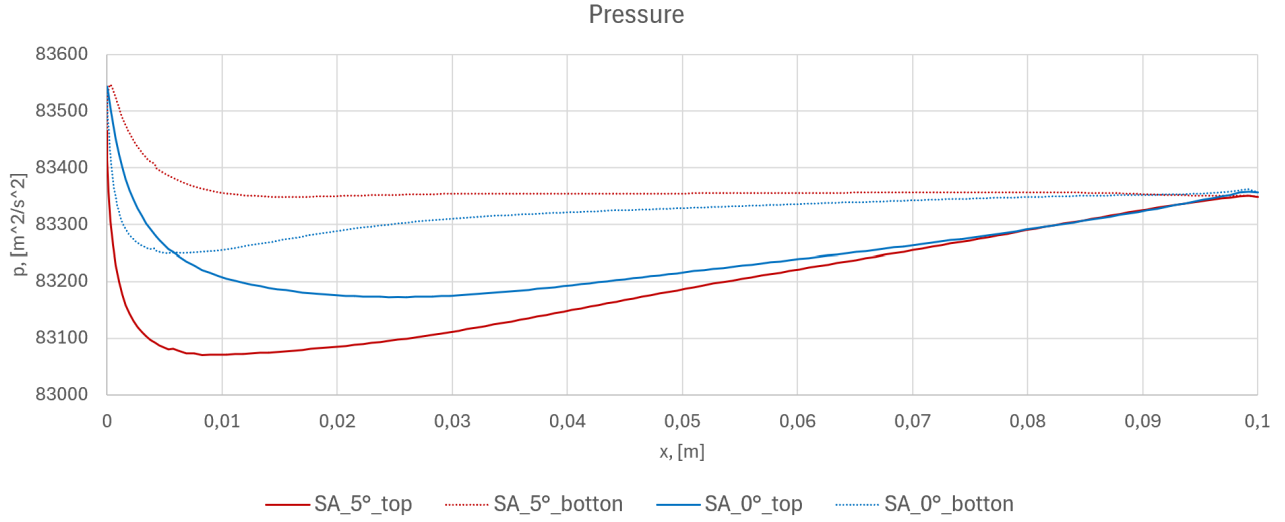


Figure 2.22: Distribution of pressure on the profile walls

The pressure distribution along the airfoil surfaces is consistent with that obtained in the previous chapter using wall-function simulations. Compared with the case at  $AOA=5^\circ$ , the simulation at  $AOA=0^\circ$  exhibits a much smaller pressure difference between the upper and lower surfaces. This behaviour is consistent with the reduction in the lift coefficient.

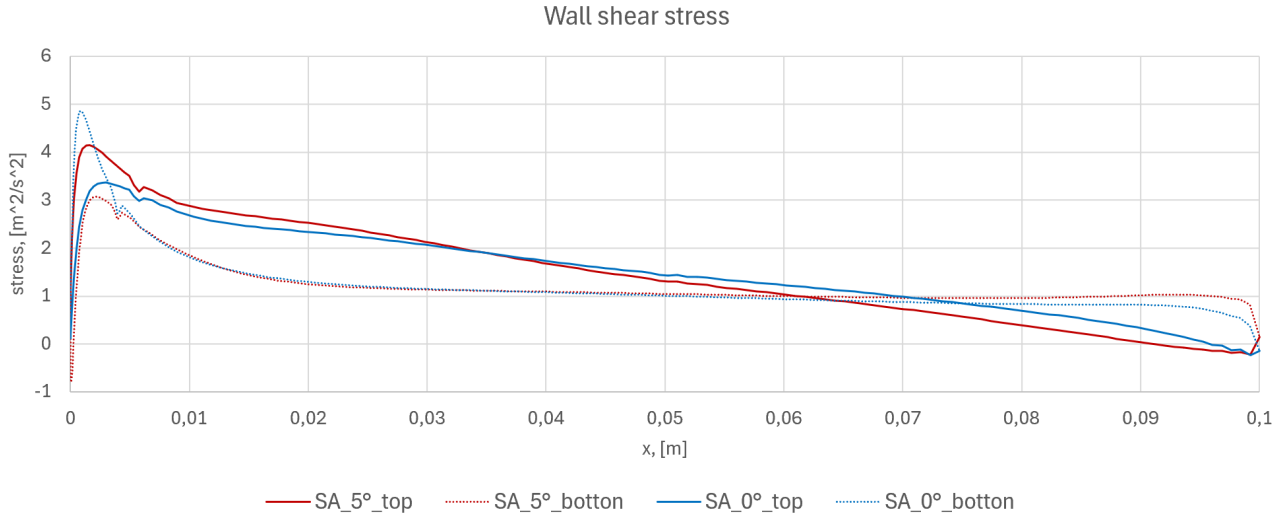


Figure 2.23: Distribution of shear on the profile walls

Regarding the wall shear stress, the values of the  $AOA = 5^\circ$  simulation are higher than the values in the same simulation with wall functions (Fig: 2.15), this behaviour confirms the more accurate results along the wall of the simulation without wall function.

About the differences between the  $AOA=5^\circ$  and  $AOA=0^\circ$  cases, the second case exhibits a lower wall shear stress on the upper surface due to the lower flow velocity over the airfoil. While, in the bottom wall, for the same reason, the shear stress is higher.

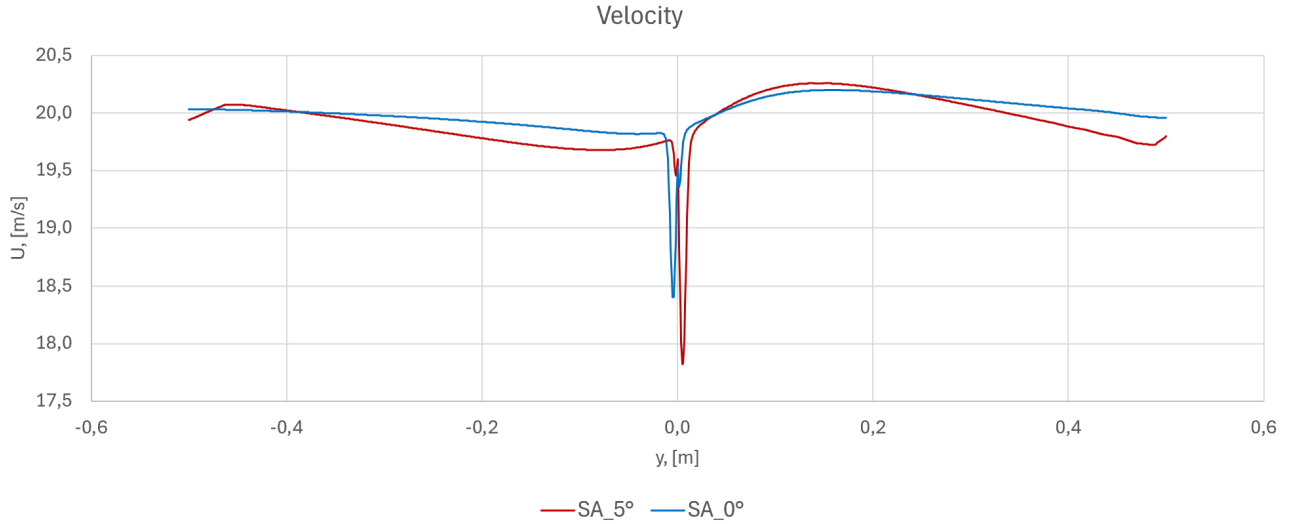


Figure 2.24: Velocity distribution in the wake of the airfoil, one chord downstream of TE

The most notable differences are the greater asymmetry of the wake and the more pronounced velocity deficit observed for the  $\text{AOA}=5^\circ$  case. This behaviour is consistent with the higher drag coefficient, which reflects the larger momentum loss in the wake.

## 2.5. Conclusions

The objective of this last part of the report is to analyze the aerodynamic behaviour of the NACA4415 airfoil through the use of different turbulence models. In particular were used a  $k-\varepsilon$  model and a SA model with wall function, and then the SA model without wall functions.

A grid independence study was first performed using three progressively refined meshes. The comparison of lift and drag coefficients, pressure distributions, wall shear stress, and wake profiles showed excellent agreement between the medium and high meshes.

The simulations carried out with the SA and  $k-\varepsilon$  turbulence models produced very similar results, confirming the robustness of the numerical setup. The pressure and wall shear stress distributions were consistent with the expected aerodynamic behaviour, while the wake analysis highlighted the higher aerodynamic loading of the  $\text{AOA} = 5^\circ$  case compared to the  $\text{AOA} = 0^\circ$  case. Finally, the wall-resolved simulations showed a good agreement with the corresponding wall-function results, providing additional confidence in the accuracy and reliability of the numerical predictions.